Four Snacks, Line by Line

C#, WPF, SQL Server and plugins: the Genetec projects explained

A walkthrough of the four Genetec snacks: what each one does, why it's built the way it is, and the code that matters, explained line by line, with the tests that check it. Interview questions and exercises close every part.

Part	What it covers
0. The big picture	How the snacks connect, the solution layout, how to read the book
1. Camera Status Monitor	WPF and MVVM, commands, the UI thread, testing ViewModels
2. Access Event Log	SQL Server schema and indexes, Dapper, reports, contract tests, .NET Framework 4.8
3. Mini Plugin Framework	Reflection, AssemblyLoadContext, fault isolation, commands, configuration
4. Web Dashboard	ASP.NET Core minimal APIs, hosted services, React and TypeScript
5. Keeping it working	Lock files, Semantic Versioning, CI on Windows and Linux, Dependabot

Every code listing shows the real file and its real line numbers. The notes under a listing refer to those numbers, so you can read the book with the project open beside it.

Contents

The big picture 4

0.1 How the snacks connect 5

0.2 The solution 5

0.3 How to read this book 6

Camera Status Monitor 7

1.1 What it does 8

1.2 MVVM in one page 8

1.3 The UI thread 9

1.4 The project files 9

1.5 Devices and status changes 10

1.6 The simulator 11

1.7 MVVM building blocks 13

1.8 A row: DeviceViewModel 14

1.9 The screen: MainViewModel 15

1.10 The app: composition without code-behind 17

1.11 The window 19

1.12 Testing the ViewModels 20

1.13 Interview questions 22

1.14 Exercises 23

Access Event Log 24

2.1 What it does 25

2.2 The schema 25

2.3 Repository, Dapper and contract tests 26

2.4 One library, two frameworks 26

2.5 The model 27

2.6 The repository interface 28

2.7 schema.sql 29

2.8 SQL Server with Dapper 30

2.9 The in-memory twin 34

2.10 The simulator 35

2.11 The ViewModel 36

2.12 The .NET Framework 4.8 app 38

2.13 The contract tests 40

2.14 Interview questions . . . 42
2.15 Exercises . . . 43

Mini Plugin Framework . . . 44

3.1 What it does . . . 45
3.2 Plugins, reflection and load contexts . . . 45
3.3 The contract . . . 46
3.4 Shared logging and configuration . . . 47
3.5 Loading a module . . . 49
3.6 Building modules into a folder . . . 51
3.7 Routing events . . . 52
3.8 The two modules . . . 54
3.9 The host program . . . 55
3.10 Testing a framework . . . 56
3.11 Interview questions . . . 57
3.12 Exercises . . . 58

Web Dashboard . . . 59

4.1 What it does . . . 60
4.2 Minimal APIs and dependency injection . . . 60
4.3 Reuse: one project reference . . . 60
4.4 Program.cs . . . 61
4.5 Filters and the demo . . . 62
4.6 The page . . . 63
4.7 Tests that control the world . . . 65
4.8 Interview questions . . . 66
4.9 Exercises . . . 67

Keeping it working . . . 68

5.1 One build for four snacks . . . 69
5.2 Lock files . . . 69
5.3 A vulnerable transitive dependency . . . 70
5.4 Semantic Versioning for a .NET solution . . . 71
5.5 The workflow . . . 71
5.6 Dependabot . . . 73
5.7 Running it all . . . 74
5.8 Interview questions . . . 74

Part 0

The big picture

Four small projects mirroring Genetec's stack: desktop security software in C# and WPF, SQL Server, frameworks that save each project from starting from scratch, and a little web.

0.1 How the snacks connect

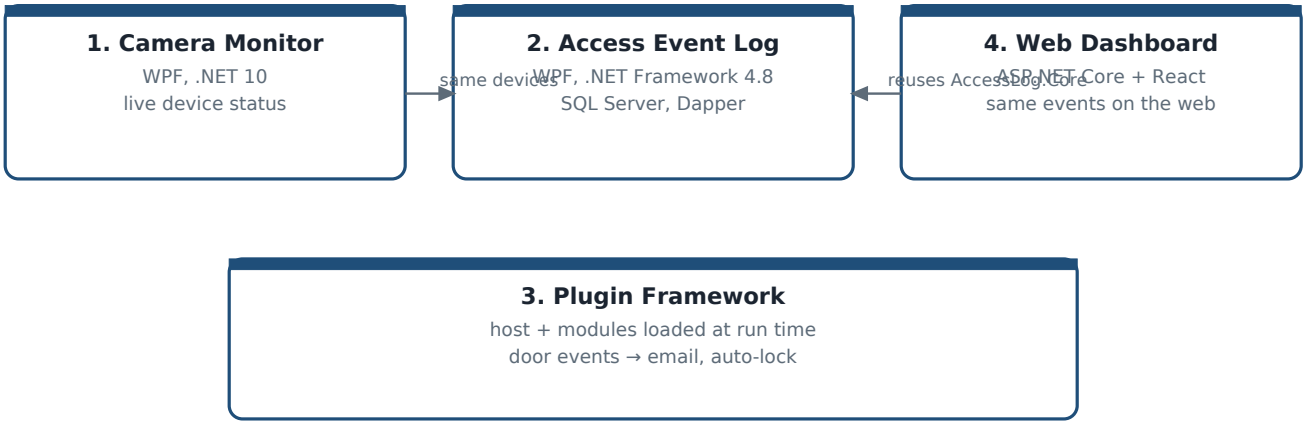


Figure 0.1. The snacks share a building: the same cameras and doors throughout.

Snack	Shows	Talking point
1. Camera Status Monitor	WPF, MVVM, commands, converters, thread hand-off, no code-behind	"I kept the view dumb so the same ViewModel could be unit-tested or reused in another plugin."
2. Access Event Log	Schema design, constraints, indexes, GROUP BY reports, Dapper, contract tests, .NET Framework 4.8	Mirrors the L3Harris grading/replay queries.
3. Mini Plugin Framework	Reflection, load contexts, a stable contract, fault isolation, commands	Connects to Unreal tools and Python build automation work.
4. Web Dashboard	Minimal APIs, DI, hosted services, React + TypeScript, API tests	Optional; reuses Snack 2 unchanged.

0.2 The solution

All four snacks are one solution, `genetec/Genetec.slnx` (the newer XML solution format), so one `dotnet build` builds everything and one `dotnet test` runs all 141 .NET tests (Snack 4's page has 9 more, run by npm). Each snack keeps the same shape: `src/` for code, `tests/` for tests.

MSBuild Genetec.slnx, lines 1–24

```
<Solution>
  <Folder Name="/snack1-camera-status-monitor/">
    <Project Path="snack1-camera-status-monitor/src/CameraMonitor.Core/CameraMonitor.Core.csproj
      " />
    <Project Path="snack1-camera-status-monitor/src/CameraMonitor.App/CameraMonitor.App.csproj"
      />
    <Project Path="snack1-camera-status-monitor/tests/CameraMonitor.Tests/CameraMonitor.Tests.cs
      proj" />
  </Folder>
  <Folder Name="/snack2-access-event-log/">
    <Project Path="snack2-access-event-log/src/AccessLog.Core/AccessLog.Core.csproj" />
    <Project Path="snack2-access-event-log/src/AccessLog.App/AccessLog.App.csproj" />
    <Project Path="snack2-access-event-log/tests/AccessLog.Tests/AccessLog.Tests.csproj" />
  </Folder>
  <Folder Name="/snack3-plugin-framework/">
    <Project Path="snack3-plugin-framework/src/SolutionFramework.Abstractions/SolutionFramework.
```

```
        Abstractions.csproj" />
    <Project Path="snack3-plugin-framework/src/SolutionFramework.Common/SolutionFramework.Common
      .csproj" />
    <Project Path="snack3-plugin-framework/src/SolutionFramework.Host/SolutionFramework.Host.csp
      roj" />
    <Project Path="snack3-plugin-framework/modules/DoorForcedEmail/DoorForcedEmail.csproj" />
    <Project Path="snack3-plugin-framework/modules/AfterHoursAutoLock/AfterHoursAutoLock.csproj"
       />
    <Project Path="snack3-plugin-framework/tests/SolutionFramework.Tests/SolutionFramework.Tests
      .csproj" />
  </Folder>
  <Folder Name="/snack4-web-dashboard/">
    <Project Path="snack4-web-dashboard/api/Dashboard.Api/Dashboard.Api.csproj" />
    <Project Path="snack4-web-dashboard/tests/Dashboard.Api.Tests/Dashboard.Api.Tests.csproj" />
  </Folder>
</Solution>
```

A consistent idea runs through every snack: **the part worth testing never depends on the part that's hard to test**. ViewModels don't reference WPF. Reports are behind an interface with an in-memory twin. The plugin host doesn't reference any module. The web API's store and clock are injected.

0.3 How to read this book

Each part opens with what the snack does and the ideas it relies on, then goes through the code file by file. Listings are the real files with real line numbers; the bold "Line n" notes underneath refer to them. Each part ends with how it's tested, interview questions with answers, and exercises.

Two ways to read it. Preparing for an interview: the opening sections of each part, the tests section, and the questions. Extending the code: read the line-by-line sections with the project open beside the book.

> The book is generated from the source by `walkthrough/build_walkthrough.py`. If you change the code, run it again (`python3 walkthrough/build_walkthrough.py`, needs `reportlab`): the line numbers are recomputed, and a listing whose anchor text has gone stops the build instead of quietly showing the wrong lines.

Part 1

Camera Status Monitor

A WPF dashboard of cameras and doors whose status changes live. The point isn't the window: it's that everything the window does lives in ViewModels that know nothing about WPF, so 28 tests can check it without ever opening a window.

1.1 What it does

The window lists nine devices from a pretend building: five cameras and four doors, each with a name, a type, a location and a status. A background simulator changes one device every two seconds: it goes online, offline or, rarely, into alarm. The row's colour follows its status (green, grey, red), the counts at the top update, and a log on the right lists the changes, newest first.

You can filter the list by typing (the filter matches name, location, type, status and id), and you can select a device in alarm and acknowledge it, which puts it back online. The Acknowledge button is only enabled when the selected device is actually in alarm.

1.2 MVVM in one page

Model-View-ViewModel splits a screen into three parts. The **View** is the XAML: layout, colours, controls. The **ViewModel** is a plain class holding the screen's state (the visible rows, the filter text, the selected device) and its actions (commands). The **Model** is the rest of the program: here, the devices and the simulator.

The View never calls the ViewModel's methods directly and the ViewModel never touches a control. They meet through **data binding**: XAML says `Text="{Binding FilterText}"` and WPF keeps the text box and the property in sync, in both directions. For WPF to know a property changed, the ViewModel raises `INotifyPropertyChanged.PropertyChanged`; for lists, `ObservableCollection` raises `CollectionChanged`. Buttons bind to `ICommand` objects, which also tell the button whether it may be clicked.

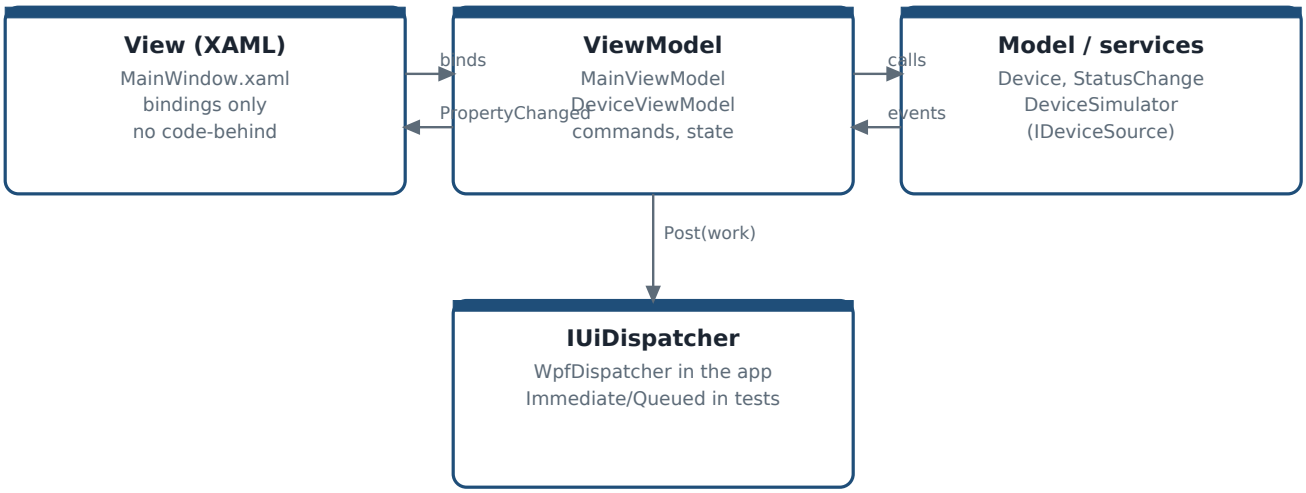


Figure 1.1. Who talks to whom. Arrows into the ViewModel are the only way in; the View only binds, and threading goes through an interface the tests can replace.

The payoff is testability. Because `ICommand`, `INotifyPropertyChanged` and `ObservableCollection` are in .NET's base library rather than WPF, the ViewModels can live in a plain `net10.0` library with no reference to WPF at all, and the tests run on any operating system, including CI's Linux machines.

1.3 The UI thread

WPF controls belong to the thread that created them, the UI thread, and throw if touched from any other. The simulator uses a `System.Threading.Timer`, whose callbacks run on thread-pool threads. If the simulator's event updated a bound property directly, WPF would either throw or, for some changes, misbehave intermittently, which is worse.

The fix is to move the work onto the UI thread. WPF's `Dispatcher` is a queue the UI thread drains between handling input and drawing; `Dispatcher.BeginInvoke` adds work to it and returns immediately. The ViewModel doesn't call the Dispatcher itself (that would tie it to WPF): it calls an `IUiDispatcher` interface, and the app supplies the WPF version.

1.4 The project files

Three projects: the core library, the WPF app and the tests. The core is the important split.

MSBuild snack1-camera-status-monitor/src/CameraMonitor.Core/CameraMonitor.Core.csproj, lines 1–12

```
1  <Project Sdk="Microsoft.NET.Sdk">
2
3    <!--
4      Everything except the window: models, the simulator and the ViewModels. It targets plain
5      net10.0 (no WPF), so the tests run on any OS and the same ViewModels could back another view
         .
6    -->
7    <PropertyGroup>
8      <TargetFramework>net10.0</TargetFramework>
9      <ImplicitUsings>enable</ImplicitUsings>
10   </PropertyGroup>
11
12 </Project>
```

Plain `net10.0`: no `UseWPF`, no Windows-only target. Anything that compiles here can't depend on WPF by accident, so the compiler enforces the architecture.

MSBuild snack1-camera-status-monitor/src/CameraMonitor.App/CameraMonitor.App.csproj, lines 1–18

```
1  <Project Sdk="Microsoft.NET.Sdk">
2
3    <!-- The WPF window. Windows only; everything it shows comes from CameraMonitor.Core. -->
4    <PropertyGroup>
5      <OutputType>WinExe</OutputType>
6      <TargetFramework>net10.0-windows</TargetFramework>
7      <UseWPF>true</UseWPF>
8      <ImplicitUsings>enable</ImplicitUsings>
9      <!-- Lets the project restore and build on Linux too (CI checks the XAML compiles there). --
         >
10     <EnableWindowsTargeting>true</EnableWindowsTargeting>
11     <AssemblyName>CameraMonitor</AssemblyName>
12   </PropertyGroup>
13
14   <ItemGroup>
15     <ProjectReference Include="..\CameraMonitor.Core\CameraMonitor.Core.csproj" />
16   </ItemGroup>
17
18 </Project>
```

Line 5 `WinExe` rather than `Exe`: no console window opens behind the app.

Line 6 The `-windows` suffix makes Windows APIs available; WPF needs it.

Line 10 Lets the project restore and compile on Linux and macOS too. It can't run there, but CI and editors on other systems can still check it builds.

Line 11 The executable is `CameraMonitor.exe` rather than the project name.

1.5 Devices and status changes

C# snack1-camera-status-monitor/src/CameraMonitor.Core/Devices/Device.cs, lines 3–27

```
 3  public enum DeviceType
 4  {
 5      Camera,
 6      Door,
 7  }
 8
 9  public enum DeviceStatus
10  {
11      Online,
12      Offline,
13      Alarm,
14  }
15
16  /// <summary>A camera or door as the simulator knows it. The UI wraps it in a DeviceViewModel.</
        summary>
17  public sealed class Device(string id, string name, DeviceType type, string location, DeviceStatu
        s status = DeviceStatus.Online)
18  {
19      public string Id { get; } = id;
20      public string Name { get; } = name;
21      public DeviceType Type { get; } = type;
22      public string Location { get; } = location;
23      public DeviceStatus Status { get; set; } = status;
24  }
25
26  /// <summary>One status change, raised by the simulator on its own thread.</summary>
27  public sealed record StatusChange(string DeviceId, DeviceStatus From, DeviceStatus To, DateTimeO
        ffset At);
```

Line 17 A **primary constructor** (C# 12): the parameters in the class header are in scope for the whole class, and here initialize get-only properties. Only `Status` can change.

Line 27 A **record**: an immutable value with equality by content, ideal for an event that says what happened. Positional syntax generates the constructor, properties, `Equals` and `ToString`.

C# snack1-camera-status-monitor/src/CameraMonitor.Core/Devices/Device.cs, lines 29–44

```
29  /// <summary>The devices in the demo building.</summary>
30  public static class SampleDevices
31  {
32      public static IReadOnlyList<Device> Create() =>
33      [
34          new("CAM-01", "Lobby entrance", DeviceType.Camera, "Ground floor"),
35          new("CAM-02", "Parking level 1", DeviceType.Camera, "P1"),
36          new("CAM-03", "Server room", DeviceType.Camera, "Floor 2"),
37          new("CAM-04", "Loading dock", DeviceType.Camera, "Ground floor"),
38          new("CAM-05", "Roof access", DeviceType.Camera, "Roof", DeviceStatus.Offline),
39          new("DOOR-01", "Main entrance", DeviceType.Door, "Ground floor"),
40          new("DOOR-02", "Server room", DeviceType.Door, "Floor 2"),
41          new("DOOR-03", "Stairwell B", DeviceType.Door, "Floor 3"),
42          new("DOOR-04", "Loading dock", DeviceType.Door, "Ground floor"),
43      ];
44  }
```

The collection expression `[ ... ]` builds the list, and `new(...)` is target-typed: the compiler knows each element is a `Device`. One camera starts offline so the counts aren't all equal at startup.

1.6 The simulator

C# snack1-camera-status-monitor/src/CameraMonitor.Core/Devices/DeviceSimulator.cs, lines 3–15

```
 3  /// <summary>Where device status changes come from: the simulator here, real hardware in a real
        product.</summary>
 4  public interface IDeviceSource : IDisposable
 5  {
 6      IReadOnlyList<Device> Devices { get; }
 7
 8      /// <summary>Raised on a background thread. Listeners that touch the UI must marshal to it.<
        /summary>
 9      event EventHandler<StatusChange>? StatusChanged;
10
11      void Start();
12
13      /// <summary>Records that an operator acknowledged an alarm: the device goes back online.</s
        ummary>
14      void Acknowledge(string deviceId);
15  }
```

The ViewModel depends on this interface, not on the simulator, so tests can raise exactly the changes they want. In a real product, the same interface would sit in front of cameras and door controllers.

C# snack1-camera-status-monitor/src/CameraMonitor.Core/Devices/DeviceSimulator.cs, lines 17–36

```
17  /// <summary>
18  /// Flips a random device's status every few seconds on a thread-pool timer, the way real device
        s
19  /// report changes whenever they like. <see cref="Tick"/> is public so tests can drive it withou
        t
20  /// waiting, and the random source can be seeded so a test sees the same changes every run.
21  /// </summary>
22  public sealed class DeviceSimulator(IReadOnlyList<Device> devices, Random random, TimeSpan inter
        val) : IDeviceSource
23  {
24      private readonly Lock _gate = new();
25      private Timer? _timer;
26
27      public DeviceSimulator() : this(SampleDevices.Create(), new Random(), TimeSpan.FromSeconds(2
        )) { }
28
29      public IReadOnlyList<Device> Devices { get; } = devices;
30
31      public event EventHandler<StatusChange>? StatusChanged;
32
33      public void Start() => _timer ??= new Timer(_ => Tick(), null, interval, interval);
34
35      /// <summary>Changes one device. Alarms are rare (1 in 6), as they should be.</summary>
36      public StatusChange Tick()
```

Line 24 `System.Threading.Lock` (.NET 9) is a dedicated lock object: a `lock` statement on it is faster and clearer than locking on a plain `object`.

Line 27 The app uses the defaults; tests pass a seeded `Random` and a long interval so the timer never fires on its own.

Line 33 `??=` only creates the timer if there isn't one, so calling `Start` twice can't start two timers. The callback runs on a thread-pool thread.

C# snack1-camera-status-monitor/src/CameraMonitor.Core/Devices/DeviceSimulator.cs, lines 35–54

```
35      /// <summary>Changes one device. Alarms are rare (1 in 6), as they should be.</summary>
36      public StatusChange Tick()
37      {
38          StatusChange change;
39          lock (_gate)
40          {
41              var device = Devices[random.Next(Devices.Count)];
42              var next = random.Next(6) switch
43              {
44                  0 => DeviceStatus.Alarm,
45                  1 or 2 => DeviceStatus.Offline,
46                  _ => DeviceStatus.Online,
47              };
48              if (next == device.Status)
49                  next = device.Status == DeviceStatus.Online ? DeviceStatus.Offline : DeviceStatu
    s.Online;
50              change = Set(device, next);
51          }
52          StatusChanged?.Invoke(this, change);
53          return change;
54      }
```

Line 42 A switch expression with patterns: 0 is alarm (1 in 6), 1 or 2 offline, anything else online. Alarms should be the rarest; a test checks they are.

Line 48 Every tick must be a real change, so picking the current status flips it instead.

Line 52 Raised *outside* the lock. Calling unknown code while holding a lock is how deadlocks start: a handler that waited for another thread holding the same lock would never return.

C# snack1-camera-status-monitor/src/CameraMonitor.Core/Devices/DeviceSimulator.cs, lines 56–66

```
56      public void Acknowledge(string deviceId)
57      {
58          StatusChange? change = null;
59          lock (_gate)
60          {
61              var device = Devices.FirstOrDefault(d => d.Id == deviceId);
62              if (device is { Status: DeviceStatus.Alarm })
63                  change = Set(device, DeviceStatus.Online);
64          }
65          if (change is not null) StatusChanged?.Invoke(this, change);
66      }
```

Acknowledging only does something for a device that's really in alarm; `device is { Status: DeviceStatus.Alarm }` is a property pattern that also checks for null. The change goes out through the same event as any other, so the window updates the same way whatever caused it.

1.7 MVVM building blocks

C# snack1-camera-status-monitor/src/CameraMonitor.Core/Mvvm/ObservableObject.cs, lines 6–22

```
/// <summary>Base class for ViewModels: tells bindings when a property changes.</summary>
public abstract class ObservableObject : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler? PropertyChanged;

    protected void OnPropertyChanged([CallerMemberName] string? name = null) =>
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));

    /// <summary>Sets the field and raises PropertyChanged, only if the value actually changed.</summary>
    protected bool SetProperty<T>(ref T field, T value, [CallerMemberName] string? name = null)
    {
        if (EqualityComparer<T>.Default.Equals(field, value)) return false;
        field = value;
        OnPropertyChanged(name);
        return true;
    }
}
```

Line 11 The compiler fills in the caller's name, so a property setter calls `SetProperty(ref _x, value)` without spelling its own name, and renaming the property can't leave a stale string.

Line 17 No change, no event. Bindings refresh on every event, so skipping no-op changes avoids needless work and update loops.

C# snack1-camera-status-monitor/src/CameraMonitor.Core/Mvvm/RelayCommand.cs, lines 5–22

```
/// <summary>
/// An ICommand made of two delegates. ICommand lives in System.ObjectModel, not WPF, so commands
/// can be created and tested without a window. WPF calls CanExecute again when
/// CanExecuteChanged fires, which enables or disables the bound button.
/// </summary>
public sealed class RelayCommand(Action execute, Func<bool>? canExecute = null) : ICommand
{
    public event EventHandler? CanExecuteChanged;

    public bool CanExecute(object? parameter) => canExecute?.Invoke() ?? true;

    public void Execute(object? parameter)
    {
        if (CanExecute(parameter)) execute();
    }

    public void RaiseCanExecuteChanged() => CanExecuteChanged?.Invoke(this, EventArgs.Empty);
}
```

A button bound to a command calls `CanExecute` to decide whether it's enabled, and again whenever `CanExecuteChanged` fires. WPF's own commands hook that event to a global "requery" signal; this one raises it explicitly, which is more predictable and works without WPF. `Execute` checks `CanExecute` again, in case it's invoked from code rather than a button.

C# snack1-camera-status-monitor/src/CameraMonitor.Core/ViewModels/IUiDispatcher.cs, lines 3–11

```
3  /// <summary>
4  /// Runs an action on the UI thread. WPF objects can only be touched from the thread that create
       d
5  /// them, and the simulator raises events on a timer thread. The app supplies a version backed b
       y
6  /// WPF's Dispatcher; tests supply one that runs actions immediately or queues them.
7  /// </summary>
8  public interface IUiDispatcher
9  {
10     void Post(Action action);
11 }
```

1.8 A row: DeviceViewModel

C# snack1-camera-status-monitor/src/CameraMonitor.Core/ViewModels/DeviceViewModel.cs, lines 6–40

```
6  /// <summary>One row in the device list.</summary>
7  public sealed class DeviceViewModel(Device device) : ObservableObject
8  {
9      private readonly Device _device = device;
10     private DeviceStatus _status = device.Status;
11     private DateTimeOffset? _lastChanged;
12
13     public string Id => _device.Id;
14     public string Name => _device.Name;
15     public DeviceType Type => _device.Type;
16     public string Location => _device.Location;
17
18     public DeviceStatus Status
19     {
20         get => _status;
21         set
22         {
23             if (SetProperty(ref _status, value)) OnPropertyChanged(nameof(IsAlarm));
24         }
25     }
26
27     public bool IsAlarm => Status == DeviceStatus.Alarm;
28
29     public DateTimeOffset? LastChanged
30     {
31         get => _lastChanged;
32         set => SetProperty(ref _lastChanged, value);
33     }
34
35     /// <summary>Does this row match the filter text? Name, location, type and status all count.
         </summary>
36     public bool Matches(string filter) =>
37         string.IsNullOrWhiteSpace(filter) ||
38         new[] { Name, Location, Id, Type.ToString(), Status.ToString() }
39             .Any(field => field.Contains(filter.Trim(), StringComparison.OrdinalIgnoreCase));
40 }
```

Line 9 The constructor parameter is copied into a field once. Using the parameter both here and in members would make the compiler warn (CS9124) that its value is captured twice, and CI treats warnings as errors.

Line 23 `IsAlarm` is computed from `Status`, so when the status changes it has to announce that `IsAlarm` may have changed too, or anything bound to it would go stale.

Line 36 The filter rule lives with the row it describes. `StringComparison.OrdinalIgnoreCase` compares without culture rules, which is right for codes and names typed by an

operator.

1.9 The screen: MainViewModel

C# snack1-camera-status-monitor/src/CameraMonitor.Core/ViewModels/MainViewModel.cs, lines 23–35

```
23      public MainViewModel(IDeviceSource source, IUiDispatcher dispatcher)
24      {
25          _source = source;
26          _dispatcher = dispatcher;
27          AllDevices = source.Devices.Select(d => new DeviceViewModel(d)).ToList();
28          _byId = AllDevices.ToDictionary(d => d.Id);
29          _acknowledgeAlarm = new RelayCommand(AcknowledgeSelected, () => SelectedDevice?.IsAlarm
          == true);
30          ClearFilterCommand = new RelayCommand(() => FilterText = "", () => FilterText.Length > 0
        );
31          ApplyFilter();
32
33          // The source raises this on its own thread; hop to the UI thread before touching anythi
          ng bound.
34          _source.StatusChanged += (_, change) => _dispatcher.Post(() => Apply(change));
35      }
```

Line 27 One `DeviceViewModel` per device, made once. `_byId` finds a row from a change in constant time.

Line 29 The button's rule, in one line: enabled only when a device is selected and it's in alarm.

Line 34 The key line for threading. The event arrives on the timer's thread; the handler only queues `Apply` for the UI thread.

C# snack1-camera-status-monitor/src/CameraMonitor.Core/ViewModels/MainViewModel.cs, lines 45–54

```
45      public string FilterText
46      {
47          get => _filterText;
48          set
49          {
50              if (!SetProperty(ref _filterText, value ?? "")) return;
51              ApplyFilter();
52              ClearFilterCommand.RaiseCanExecuteChanged();
53          }
54      }
```

Typing in the filter box sets `FilterText` on every keystroke (the XAML asks for `UpdateSourceTrigger=PropertyChanged`). The setter re-filters and tells the Clear button it may now be enabled. Changing the selection re-checks the Acknowledge button the same way.

C# snack1-camera-status-monitor/src/CameraMonitor.Core/ViewModels/MainViewModel.cs, lines 69–71

```
69      public int OnlineCount => AllDevices.Count(d => d.Status == DeviceStatus.Online);
70      public int OfflineCount => AllDevices.Count(d => d.Status == DeviceStatus.Offline);
71      public int AlarmCount => AllDevices.Count(d => d.Status == DeviceStatus.Alarm);
```

Computed properties with no field: they're cheap to recount for nine devices, and they can never disagree with the rows. The ViewModel announces them when a change arrives.

C# snack1-camera-status-monitor/src/CameraMonitor.Core/ViewModels/MainViewModel.cs, lines 80–97

```
80      /// <summary>Applies one change from the source. Always called on the UI thread.</summary>
81      private void Apply(StatusChange change)
82      {
83          if (!_byId.TryGetValue(change.DeviceId, out var device)) return;
84          device.Status = change.To;
85          device.LastChanged = change.At;
86
87          RecentChanges.Insert(0, $"{change.At:HH:mm:ss}  {device.Name} ({device.Id}): {change.Fro
            m} → {change.To}");
88          while (RecentChanges.Count > MaxRecentChanges) RecentChanges.RemoveAt(RecentChanges.Coun
            t - 1);
89
90          OnPropertyChanged(nameof(OnlineCount));
91          OnPropertyChanged(nameof(OfflineCount));
92          OnPropertyChanged(nameof(AlarmCount));
93          _acknowledgeAlarm.RaiseCanExecuteChanged();
94
95          // A status change can move a device in or out of a filter like "alarm".
96          if (FilterText.Length > 0) ApplyFilter();
97      }
```

Line 87 Newest first, and trimmed to `MaxRecentChanges` so a long-running window doesn't grow without bound.

Line 61 If the selected device just went into alarm (or out of it), the button's answer changed even though the selection didn't.

Line 96 A filter like "alarm" matches on status, so a status change can move a device into or out of the filtered list.

C# snack1-camera-status-monitor/src/CameraMonitor.Core/ViewModels/MainViewModel.cs, lines 99–111

```
99      /// <summary>Updates VisibleDevices in place, so the list keeps its selection and scroll pos
          ition.</summary>
100     private void ApplyFilter()
101     {
102         var wanted = AllDevices.Where(d => d.Matches(FilterText)).ToList();
103         for (var i = VisibleDevices.Count - 1; i >= 0; i--)
104             if (!wanted.Contains(VisibleDevices[i])) VisibleDevices.RemoveAt(i);
105         for (var i = 0; i < wanted.Count; i++)
106         {
107             if (i < VisibleDevices.Count && VisibleDevices[i] == wanted[i]) continue;
108             VisibleDevices.Insert(i, wanted[i]);
109         }
110         if (SelectedDevice is not null && !VisibleDevices.Contains(SelectedDevice)) SelectedDevi
            ce = null;
111     }
```

The obvious version, clearing `VisibleDevices` and adding the matches, would make the ListView drop its selection and jump to the top on every status change. Instead the method removes rows that no longer match and inserts the ones that now do, at the position they have in `AllDevices`. The list ends up in the original order, and rows that stay never leave the collection. WPF's usual tool for this, `CollectionViewSource`, is a WPF type, so using it would pull WPF back into the core.

1.10 The app: composition without code-behind

XAML snack1-camera-status-monitor/src/CameraMonitor.App/App.xaml, lines 1–15

```xaml
<Application x:Class="CameraMonitor.App.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             xmlns:services="clr-namespace:CameraMonitor.App.Services"
             xmlns:converters="clr-namespace:CameraMonitor.App.Converters"
             StartupUri="MainWindow.xaml">
    <Application.Resources>
        <!--
          The composition root lives in XAML: the locator builds the simulator and the ViewModel
        ,
          and MainWindow binds its DataContext to it. That's how the code-behind files stay empt
        y.
          -->
        <services:ViewModelLocator x:Key="Locator" />
        <converters:StatusToBrushConverter x:Key="StatusBrush" />
    </Application.Resources>
</Application>
```

There's no `App.xaml.cs` at all: WPF generates the `App` class and its `Main` from the XAML. Resources declared here are created when the app starts and can be found from any window with `StaticResource`.

C# snack1-camera-status-monitor/src/CameraMonitor.App/Services/ViewModelLocator.cs, lines 1–20

```csharp
using CameraMonitor.Core.Devices;
using CameraMonitor.Core.ViewModels;

namespace CameraMonitor.App.Services;

/// <summary>
/// Builds the object graph once and hands the window its ViewModel. Declared as a resource in
/// App.xaml, so no code-behind is needed to wire anything up.
/// </summary>
public sealed class ViewModelLocator
{
    private readonly Lazy<MainViewModel> _main = new(() =>
    {
        var viewModel = new MainViewModel(new DeviceSimulator(), new WpfDispatcher());
        viewModel.Start();
        return viewModel;
    });

    public MainViewModel Main => _main.Value;
}
```

The **composition root**: the one place that knows which concrete classes are used. `Lazy<T>` creates the ViewModel the first time the window asks for it, by which point WPF's `Application` exists and `WpfDispatcher` can capture its Dispatcher. A bigger app would use a dependency injection container here; for one window, a locator is clearer.

C# snack1-camera-status-monitor/src/CameraMonitor.App/Services/WpfDispatcher.cs, lines 1–13

```csharp
using System.Windows;
using System.Windows.Threading;
using CameraMonitor.Core.ViewModels;

namespace CameraMonitor.App.Services;

/// <summary>IUiDispatcher backed by WPF: queues the action on the UI thread's message loop.</su
```

```
        mmary>
 8  public sealed class WpfDispatcher : IUiDispatcher
 9  {
10      private readonly Dispatcher _dispatcher = Application.Current.Dispatcher;
11
12      public void Post(Action action) => _dispatcher.BeginInvoke(action, DispatcherPriority.Normal
          );
13  }
```

`BeginInvoke` queues and returns; `Invoke` would block the timer thread until the UI thread ran the work, which can deadlock if the UI thread is ever waiting on that timer.

C# snack1-camera-status-monitor/src/CameraMonitor.App/Converters/StatusToBrushConverter.cs, lines 1–33

```
 1  using System.Globalization;
 2  using System.Windows.Data;
 3  using System.Windows.Media;
 4  using CameraMonitor.Core.Devices;
 5
 6  namespace CameraMonitor.App.Converters;
 7
 8  /// <summary>Device status to colour: green Online, grey Offline, red Alarm.</summary>
 9  [ValueConversion(typeof(DeviceStatus), typeof(Brush))]
10  public sealed class StatusToBrushConverter : IValueConverter
11  {
12      private static readonly Brush Online = Freeze(new SolidColorBrush(Color.FromRgb(0x2E, 0x9E,
          0x4F)));
13      private static readonly Brush Offline = Freeze(new SolidColorBrush(Color.FromRgb(0x8A, 0x93,
           0x9B)));
14      private static readonly Brush Alarm = Freeze(new SolidColorBrush(Color.FromRgb(0xD6, 0x33, 0
          x33)));
15
16      public object Convert(object? value, Type targetType, object? parameter, CultureInfo culture
          ) =>
17          value switch
18          {
19              DeviceStatus.Online => Online,
20              DeviceStatus.Alarm => Alarm,
21              _ => Offline,
22          };
23
24      public object ConvertBack(object? value, Type targetType, object? parameter, CultureInfo cul
          ture) =>
25          throw new NotSupportedException("Status colours are display-only.");
26
27      // Frozen brushes are immutable, so every row can share one, from any thread.
28      private static Brush Freeze(Brush brush)
29      {
30          brush.Freeze();
31          return brush;
32      }
33  }
```

Line 9 Documents the types for designers and tools; WPF itself doesn't require it.

Line 12 A frozen brush is immutable and thread-safe, so one instance serves every row. Unfrozen WPF objects carry change-tracking overhead.

Line 25 Colours never flow back into the ViewModel, so converting back is a bug worth hearing about.

1.11 The window

XAML snack1-camera-status-monitor/src/CameraMonitor.App/MainWindow.xaml, lines 1–6

```
<Window x:Class="CameraMonitor.App.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Camera Status Monitor" Width="900" Height="560" MinWidth="640" MinHeight="400"
        DataContext="{Binding Main, Source={StaticResource Locator}}"
        Background="#F6F7F9" FontSize="13">
```

The `DataContext` binding is the whole wiring: every binding inside the window is relative to the `MainViewModel` the locator returns.

XAML snack1-camera-status-monitor/src/CameraMonitor.App/MainWindow.xaml, lines 20–32

```
        <!-- Counts -->
        <StackPanel Grid.ColumnSpan="3" Orientation="Horizontal" Margin="0,0,0,12">
            <TextBlock Text="Camera Status Monitor" FontSize="20" FontWeight="SemiBold" Margin="
    0,0,24,0" />
            <TextBlock VerticalAlignment="Center" Margin="0,0,16,0" Foreground="#2E9E4F">
                <Run Text="● Online " /><Run Text="{Binding OnlineCount, Mode=OneWay}" FontWeigh
    t="SemiBold" />
            </TextBlock>
            <TextBlock VerticalAlignment="Center" Margin="0,0,16,0" Foreground="#8A939B">
                <Run Text="● Offline " /><Run Text="{Binding OfflineCount, Mode=OneWay}" FontWei
    ght="SemiBold" />
            </TextBlock>
            <TextBlock VerticalAlignment="Center" Foreground="#D63333">
                <Run Text="● Alarm " /><Run Text="{Binding AlarmCount, Mode=OneWay}" FontWeight=
    "SemiBold" />
            </TextBlock>
        </StackPanel>
```

A `Run` is a piece of text inside a `TextBlock`. `Mode=OneWay` is needed because `Run.Text` binds two-way by default, and the counts are read-only.

XAML snack1-camera-status-monitor/src/CameraMonitor.App/MainWindow.xaml, lines 34–42

```
        <!-- Filter and acknowledge -->
        <DockPanel Grid.Row="1" Grid.Column="0" Margin="0,0,0,8" LastChildFill="True">
            <Button DockPanel.Dock="Right" Padding="12,4" Margin="8,0,0,0"
                    Content="Acknowledge alarm" Command="{Binding AcknowledgeAlarmCommand}" />
            <Button DockPanel.Dock="Right" Padding="10,4" Margin="8,0,0,0"
                    Content="Clear" Command="{Binding ClearFilterCommand}" />
            <TextBox Text="{Binding FilterText, UpdateSourceTrigger=PropertyChanged}" Padding="6
    ,4"
                     ToolTip="Filter by name, location, type or status (try &amp;quot;alarm&amp;quot;)"
    />
        </DockPanel>
```

XAML snack1-camera-status-monitor/src/CameraMonitor.App/MainWindow.xaml, lines 44–67

```
        <!-- Devices -->
        <ListView Grid.Row="2" Grid.Column="0" ItemsSource="{Binding VisibleDevices}"
                  SelectedItem="{Binding SelectedDevice}" SelectionMode="Single">
            <ListView.View>
                <GridView>
                    <GridViewColumn Header="Status" Width="110">
                        <GridViewColumn.CellTemplate>
                            <DataTemplate>
                                <StackPanel Orientation="Horizontal">
                                    <Ellipse Width="10" Height="10" Margin="0,0,6,0"
```

```
                                            Fill="{Binding Status, Converter={StaticResource St
atusBrush}}" />
                                <TextBlock Text="{Binding Status}"
                                           Foreground="{Binding Status, Converter={StaticRes
ource StatusBrush}}" />
                            </StackPanel>
                        </DataTemplate>
                    </GridViewColumn.CellTemplate>
                </GridViewColumn>
                <GridViewColumn Header="Name" Width="170" DisplayMemberBinding="{Binding Nam
e}" />
                <GridViewColumn Header="Type" Width="80" DisplayMemberBinding="{Binding Type
}" />
                <GridViewColumn Header="Location" Width="110" DisplayMemberBinding="{Binding
 Location}" />
                <GridViewColumn Header="Id" Width="80" DisplayMemberBinding="{Binding Id}" /
>
            </GridView>
        </ListView.View>
    </ListView>
```

Line 46 Two-way: clicking a row sets `SelectedDevice`, and the ViewModel clearing it (when the filter hides it) clears the highlight.

Line 54 Each cell's `DataContext` is its `DeviceViewModel`, so `Status` here is the row's status.

C# snack1-camera-status-monitor/src/CameraMonitor.App/MainWindow.xaml.cs, lines 1–7

```
namespace CameraMonitor.App;

/// <summary>No code here on purpose: everything the window does is bound to MainViewModel.</sum
mary>
public partial class MainWindow
{
    public MainWindow() => InitializeComponent();
}
```

1.12 Testing the ViewModels

The test project references only the core. Two fake dispatchers make threading testable, and a fake device source raises exactly the changes a test asks for.

C# snack1-camera-status-monitor/tests/CameraMonitor.Tests/Fakes.cs, lines 6–10

```
/// <summary>Runs every posted action straight away, as if the test were the UI thread.</summary
>
public sealed class ImmediateDispatcher : IUiDispatcher
{
    public void Post(Action action) => action();
}
```

C# snack1-camera-status-monitor/tests/CameraMonitor.Tests/Fakes.cs, lines 12–25

```
/// <summary>Holds posted actions until <see cref="RunAll"/>, like a UI thread that's busy.</sum
mary>
public sealed class QueuedDispatcher : IUiDispatcher
{
    private readonly Queue<Action> _pending = new();

    public int Pending => _pending.Count;

    public void Post(Action action) => _pending.Enqueue(action);

```

```
    public void RunAll()
    {
        while (_pending.TryDequeue(out var action)) action();
    }
}
```

C# snack1-camera-status-monitor/tests/CameraMonitor.Tests/MainViewModelTests.cs, lines 51–69

```
    public void Changes_from_the_device_thread_wait_for_the_ui_thread()
    {
        // The simulator raises events on a timer thread. Nothing bound to the UI may change unt
        il
        // the dispatcher runs the work on the UI thread.
        var source = FakeDeviceSource.Building();
        var dispatcher = new QueuedDispatcher();
        var vm = new MainViewModel(source, dispatcher);

        source.Raise("CAM-01", DeviceStatus.Offline);

        Assert.Equal(1, dispatcher.Pending);
        Assert.Equal(DeviceStatus.Online, Row(vm, "CAM-01").Status);
        Assert.Empty(vm.RecentChanges);

        dispatcher.RunAll();

        Assert.Equal(DeviceStatus.Offline, Row(vm, "CAM-01").Status);
        Assert.Single(vm.RecentChanges);
    }
```

This is the threading guarantee as a test: after the source raises a change, nothing bound has changed, and one action is waiting. Only when the "UI thread" runs the queue does the row update. If someone later "simplified" the ViewModel to apply changes directly, this test would fail.

C# snack1-camera-status-monitor/tests/CameraMonitor.Tests/MainViewModelTests.cs, lines 86–96

```
    [InlineData("server", new[] { "DOOR-02" })]
    [InlineData("GROUND", new[] { "CAM-01", "DOOR-01" })]
    [InlineData("camera", new[] { "CAM-01", "CAM-02" })]
    [InlineData("alarm", new[] { "DOOR-02" })]
    [InlineData("  door-01 ", new[] { "DOOR-01" })]
    [InlineData("nothing like this", new string[0])]
    public void Filter_matches_name_location_type_status_and_id_ignoring_case(string filter, str
      ing[] expected)
    {
        var (vm, _) = Create();
        vm.FilterText = filter;
        Assert.Equal(expected, vm.VisibleDevices.Select(d => d.Id));
```

One theory, six cases: each `[InlineData]` is a separate test in the results. The cases cover every field the filter reads, case-insensitivity, surrounding spaces and no match at all.

C# snack1-camera-status-monitor/tests/CameraMonitor.Tests/MainViewModelTests.cs, lines 136–146

```
    public void Acknowledge_is_only_enabled_for_a_selected_device_in_alarm()
    {
        var (vm, _) = Create();
        Assert.False(vm.AcknowledgeAlarmCommand.CanExecute(null), "nothing selected");

        vm.SelectedDevice = Row(vm, "CAM-01");
        Assert.False(vm.AcknowledgeAlarmCommand.CanExecute(null), "online device");

        vm.SelectedDevice = Row(vm, "DOOR-02");
        Assert.True(vm.AcknowledgeAlarmCommand.CanExecute(null), "device in alarm");
```

```
146     }
```

C# snack1-camera-status-monitor/tests/CameraMonitor.Tests/DeviceSimulatorTests.cs, lines 11–25

```
11      public void Every_tick_changes_exactly_one_device_to_a_different_status()
12      {
13          var simulator = Seeded();
14          for (var i = 0; i < 200; i++)
15          {
16              var before = simulator.Devices.ToDictionary(d => d.Id, d => d.Status);
17
18              var change = simulator.Tick();
19
20              Assert.NotEqual(change.From, change.To);
21              Assert.Equal(before[change.DeviceId], change.From);
22              var changed = simulator.Devices.Where(d => d.Status != before[d.Id]).ToList();
23              Assert.Equal(change.DeviceId, Assert.Single(changed).Id);
24          }
25      }
```

The simulator's contract, checked over 200 ticks: exactly one device changes, to a different status, and the change it reports matches what happened. The seeded `Random` makes a failure reproducible.

Test class	Tests	What it pins down
MainViewModelTests	18	starting state, changes, threading, filter (six cases), clear, selection, acknowledge, the log cap
DeviceSimulatorTests	7	one change per tick, same seed same changes, alarms rarest, events, acknowledge, the timer
RelayCommandTests	3	execute, the condition, CanExecuteChanged

1.13 Interview questions

Why is there no code in MainWindow.xaml.cs?

Anything in code-behind can only be tested by creating the window. With all state and behaviour in the ViewModel, the window is a description of layout and bindings, and the ViewModel is tested directly. The same ViewModel could also back a different view, which is the talking point.

The simulator raises events on a timer thread. What goes wrong if the ViewModel updates directly?

WPF objects have thread affinity. Changing an `ObservableCollection` bound to a ListView from another thread throws a `NotSupportedException`; property changes may appear to work and fail later. The ViewModel posts every change to the UI thread through `IUiDispatcher`.

Why an IUiDispatcher interface instead of calling Application.Current.Dispatcher?

It keeps WPF out of the ViewModel (so the core library has no WPF reference and tests run anywhere) and makes the threading behaviour testable: a queued fake proves nothing changes until the UI thread runs.

When does a button bound to a command re-evaluate whether it's enabled?

When the command raises `CanExecuteChanged`. This ViewModel raises it when the selection changes and when a status change arrives, because either can change whether acknowledging makes sense.

Why update VisibleDevices in place instead of replacing it?

Replacing or clearing the collection resets the ListView: the selection is lost and it scrolls to the top, on every status change. Removing and inserting individual rows keeps both.

Why freeze the brushes?

A frozen Freezable can't change, so WPF skips change tracking and can share it across rows and threads.

1.14 Exercises

Alarm sound

Play a sound when any device goes into alarm, without putting audio code in the ViewModel.

Hint. Add an `IAlarmNotifier` the ViewModel calls; implement it with `SystemSounds` in the app and with a counting fake in a test.

Sorting

Let the operator sort by clicking a column header (status first, then name).

Hint. Keep sorting in the ViewModel (a `SortBy` property) and extend `ApplyFilter` to insert in sorted order; test that a status change re-sorts.

Acknowledged, not cleared

Real systems keep an acknowledged alarm visible until it's resolved. Add an Acknowledged state with its own colour.

Hint. Add a status and a converter case; decide what the simulator does next to an acknowledged device, and test the transitions.

Stress

Run the simulator every 10 ms for a minute and watch memory and responsiveness.

Hint. Check the change log stays capped; consider batching changes that arrive within one dispatcher tick.

Part 2

Access Event Log

Snack 1's events, saved and searchable: a SQL Server schema with constraints and indexes, two reports written as real SQL, a repository with a Dapper implementation and an in-memory one held to the same contract, and a WPF app on .NET Framework 4.8.

2.1 What it does

Every badge swipe and every camera status change is recorded. The Events tab lists them newest first, with a search box (door, camera, location or cardholder) and an outcome filter (granted, denied, status changes). The Reports tab answers two questions an operator actually asks: *which doors denied access in the last 24 hours, and how often*, and *which five devices are the noisiest*. A simulator records a new event every 1.5 seconds, so both tabs move while you watch.

The app stores events in SQL Server when it can reach one, and falls back to an in-memory store when it can't, saying which in the status bar.

2.2 The schema

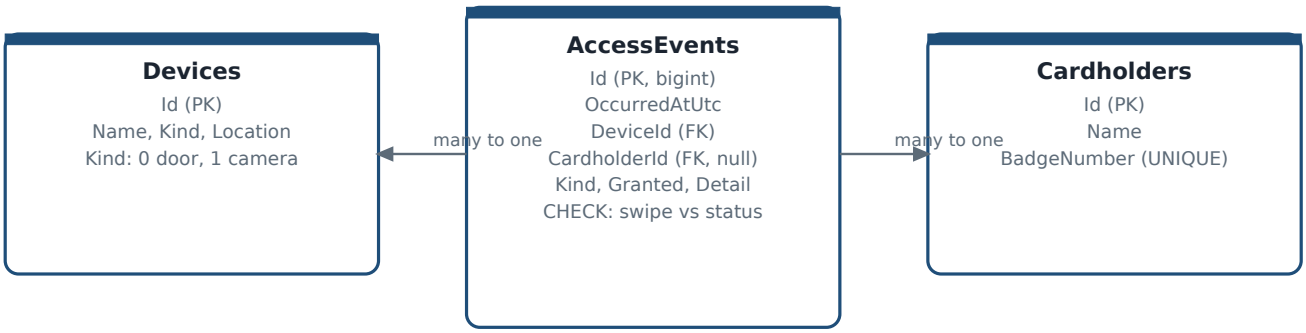


Figure 2.1. Three tables. An event always has a device; only badge swipes have a cardholder.

Constraints make bad data impossible rather than unlikely. Foreign keys stop an event pointing at a device that doesn't exist; a unique constraint stops two people having the same badge; a `CHECK` constraint states that a badge swipe has a cardholder and a decision while a status change has neither. The application checks too, but the database is the last line of defence, and it protects against every client, not just this one.

Indexes are the other half. Without one, "denied in the last 24 hours" reads every row ever recorded. An index is a sorted copy of some columns that lets SQL Server jump to the start of the time range and read only what's in it (a *seek* instead of a *scan*). If the index also *includes* the other columns the query needs, SQL Server never visits the table at all: a **covering index**.

2.3 Repository, Dapper and contract tests

A **repository** is an interface describing storage in the application's terms (record an event, search, run a report) rather than the database's. Code above it doesn't know where data lives, so the same ViewModel runs on SQL Server or in memory, and tests use the in-memory one.

Dapper is a micro-ORM: you write the SQL, it opens nothing for you, and it maps result columns onto object properties by name. That suits this snack, whose point is the SQL. Entity Framework would generate the SQL instead (and EF Core doesn't run on .NET Framework).

Two implementations of one interface can drift: the in-memory one might sort differently or forget a rule. **Contract tests** prevent that. The tests are written once, against the interface, in an abstract base class; one subclass runs them on the in-memory store, another on a real SQL Server. If they disagree, a test fails.

2.4 One library, two frameworks

MSBuild snack2-access-event-log/src/AccessLog.Core/AccessLog.Core.csproj, lines 1–30

```
1  <Project Sdk="Microsoft.NET.Sdk">
2
3    <!--
4      The model, the repository (SQL Server and in-memory), the simulator and the ViewModels.
5      net48 for the WPF app the README asks for; net10.0 for the tests and Snack 4's web API.
6    -->
7    <PropertyGroup>
8      <TargetFrameworks>net48;net10.0</TargetFrameworks>
9    </PropertyGroup>
10
11   <ItemGroup>
12     <PackageReference Include="Dapper" Version="2.1.89" />
13     <PackageReference Include="Microsoft.Data.SqlClient" Version="7.1.1" />
14     <!--
15       SqlClient depends on System.Security.Cryptography.Pkcs, and the versions it asks for have
16       known vulnerabilities. Referencing a patched version directly overrides the transitive one
         .
17     -->
18     <PackageReference Include="System.Security.Cryptography.Pkcs" Version="10.0.15" />
19   </ItemGroup>
20
21   <!-- Lets net48 build anywhere (including Linux CI) without the .NET Framework targeting pack.
          -->
22   <ItemGroup Condition="'$(TargetFramework)' == 'net48'">
23     <PackageReference Include="Microsoft.NETFramework.ReferenceAssemblies" Version="1.0.3" Priva
         teAssets="all" />
24   </ItemGroup>
25
26   <ItemGroup>
27     <EmbeddedResource Include="Data\schema.sql" />
28   </ItemGroup>
29
30 </Project>
```

Line 8 Plural `TargetFrameworks`: the project compiles twice. `net48` for the WPF app the README asks for; `net10.0` for the tests and for Snack 4's web API. The code may only use what both frameworks have, so there's no `string.Contains(string, StringComparison)` or `Lock` in this project.

Line 15 A **transitive dependency override**. SqlClient depends on Pkcs; the versions it asked for had known vulnerabilities, so CI's restore warned. Referencing a patched version directly makes NuGet pick it. Part 5 tells the story.

Line 23 The .NET Framework's reference assemblies as a package, so `net48` compiles on machines (and CI runners) without the targeting pack installed. `PrivateAssets="all"` keeps it out of anything that references this project.

Line 27 The schema ships inside the DLL, so the app can create its database wherever it runs, with no file to deploy alongside.

2.5 The model

C# snack2-access-event-log/src/AccessLog.Core/Model/Model.cs, lines 11–19

```
11  /// <summary>What happened at a device. Stored as a number in AccessEvents.Kind.</summary>
12  public enum EventKind
13  {
14      /// <summary>A cardholder presented a badge. <see cref="AccessEvent.Granted"/> says what the
        door decided.</summary>
15      BadgeSwipe = 0,
16
17      /// <summary>A device went online, offline or into alarm (Snack 1's status changes).</summar
        y>
18      StatusChange = 1,
19  }
```

C# snack2-access-event-log/src/AccessLog.Core/Model/Model.cs, lines 38–55

```
38  /// <summary>A row of the AccessEvents table. Times are always UTC.</summary>
39  public sealed class AccessEvent
40  {
41      public long Id { get; set; }
42      public DateTime OccurredAtUtc { get; set; }
43      public int DeviceId { get; set; }
44
45      /// <summary>Who swiped. Null for status changes.</summary>
46      public int? CardholderId { get; set; }
47
48      public EventKind Kind { get; set; }
49
50      /// <summary>Access granted (true) or denied (false). Null for status changes.</summary>
51      public bool? Granted { get; set; }
52
53      /// <summary>Free text, e.g. "Online → Alarm" for a status change.</summary>
54      public string? Detail { get; set; }
55  }
```

Plain classes with settable properties, not records: Dapper fills them by setting properties, and records with `init` setters need a compiler helper type that .NET Framework doesn't have. `Granted` is `bool?` because a status change has no decision: null means "not applicable", which a plain `bool` can't say.

C# snack2-access-event-log/src/AccessLog.Core/Model/Model.cs, lines 57–78

```
57  /// <summary>An event joined with its device and cardholder, as the event list shows it.</summar
        y>
58  public sealed class AccessEventView
59  {
60      public long Id { get; set; }
```

```
61      public DateTime OccurredAtUtc { get; set; }
62      public int DeviceId { get; set; }
63      public string DeviceName { get; set; } = "";
64      public string Location { get; set; } = "";
65      public int? CardholderId { get; set; }
66      public string? CardholderName { get; set; }
67      public EventKind Kind { get; set; }
68      public bool? Granted { get; set; }
69      public string? Detail { get; set; }
70
71      /// <summary>"Granted", "Denied" or the status change, for display.</summary>
72      public string Outcome => Granted switch
73      {
74          true => "Granted",
75          false => "Denied",
76          null => Detail ?? "",
77      };
78  }
```

Line 72 A switch on `bool?` covers all three cases, so the list and the web page show the same words without either deciding.

C# snack2-access-event-log/src/AccessLog.Core/Model/Model.cs, lines 98–113

```
98  /// <summary>Which events to list. Every filter is optional.</summary>
99  public sealed class EventQuery
100 {
101     public DateTime? FromUtc { get; set; }
102     public DateTime? ToUtc { get; set; }
103     public int? DeviceId { get; set; }
104     public int? CardholderId { get; set; }
105     public EventKind? Kind { get; set; }
106     public bool? Granted { get; set; }
107
108     /// <summary>Matches device name, location or cardholder name (case-insensitive).</summary>
109     public string? Text { get; set; }
110
111     /// <summary>Newest events first, at most this many.</summary>
112     public int Limit { get; set; } = 200;
113 }
```

One object carries every filter, all optional. Adding a filter later is a new property, not a new method on the interface, so neither implementation's callers break.

2.6 The repository interface

C# snack2-access-event-log/src/AccessLog.Core/Data/IAccessEventRepository.cs, lines 8–31

```
8   /// <summary>
9   /// Everything the app needs from storage. Two implementations: SQL Server (Dapper) for the real
10  /// thing, in-memory for tests and for running without a database. The same contract tests run
11  /// against both, so they behave the same.
12  /// </summary>
13  public interface IAccessEventRepository
14  {
15      Task<int> AddDeviceAsync(Device device);
16      Task<int> AddCardholderAsync(Cardholder cardholder);
17      Task<IReadOnlyList<Device>> GetDevicesAsync();
18      Task<IReadOnlyList<Cardholder>> GetCardholdersAsync();
19
20      /// <summary>Saves the event and returns its new id. The device (and cardholder, if any) mus
          t exist.</summary>
21      Task<long> RecordAsync(AccessEvent accessEvent);
22
```

```
    /// <summary>Events matching the query, newest first, at most <see cref="EventQuery.Limit"/>
      .</summary>
    Task<IReadOnlyList<AccessEventView>> SearchAsync(EventQuery query);

    /// <summary>Denied badge swipes per door since a time, most denials first. Doors with none
      are left out.</summary>
    Task<IReadOnlyList<DeniedByDoorRow>> DeniedByDoorAsync(DateTime sinceUtc);

    /// <summary>The devices with the most events of any kind since a time, most first, at most
      <paramref name="top"/>.</summary>
    Task<IReadOnlyList<NoisyDeviceRow>> NoisiestDevicesAsync(DateTime sinceUtc, int top);
}
```

Everything is `async`: database calls wait on the network, and an async method lets the UI thread keep drawing meanwhile. The comments are the contract: "newest first", "doors with none are left out", "most first". The contract tests check exactly these sentences.

C# snack2-access-event-log/src/AccessLog.Core/Data/IAccessEventRepository.cs, lines 33–34

```
/// <summary>A reference to something that doesn't exist, e.g. an event for an unknown device.</
    summary>
public sealed class UnknownReferenceException(string message) : Exception(message);
```

Both implementations translate their own failure (a SQL foreign-key error, a missing list entry) into this one exception, so callers handle one thing.

2.7 schema.sql

SQL snack2-access-event-log/src/AccessLog.Core/Data/schema.sql, lines 3–9

```
IF OBJECT_ID(N'dbo.Devices', N'U') IS NULL
CREATE TABLE dbo.Devices (
    Id       INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Devices PRIMARY KEY,
    Name     NVARCHAR(100) NOT NULL,
    Kind     TINYINT       NOT NULL CONSTRAINT CK_Devices_Kind CHECK (Kind IN (0, 1)), -- 0 door
      , 1 camera
    Location NVARCHAR(100) NOT NULL
);
```

Line 3 **Idempotent**: safe to run on every start. `OBJECT_ID` returns null if no user table (`'U'`) has that name.

Line 7 The enum's values, enforced. Naming constraints (`CK_...`) makes error messages and later changes readable.

SQL snack2-access-event-log/src/AccessLog.Core/Data/schema.sql, lines 18–30

```
IF OBJECT_ID(N'dbo.AccessEvents', N'U') IS NULL
CREATE TABLE dbo.AccessEvents (
    Id            BIGINT IDENTITY(1,1) NOT NULL CONSTRAINT PK_AccessEvents PRIMARY KEY,
    OccurredAtUtc DATETIME2(3)  NOT NULL,
    DeviceId      INT           NOT NULL CONSTRAINT FK_AccessEvents_Devices REFERENCES dbo.Devic
      es (Id),
    CardholderId  INT           NULL     CONSTRAINT FK_AccessEvents_Cardholders REFERENCES dbo.C
      ardholders (Id),
    Kind          TINYINT       NOT NULL, -- 0 badge swipe, 1 status change
    Granted       BIT           NULL,     -- null for status changes
    Detail        NVARCHAR(200) NULL,
    -- A badge swipe always has a cardholder and a decision; a status change has neither.
    CONSTRAINT CK_AccessEvents_Shape CHECK (
        (Kind = 0 AND CardholderId IS NOT NULL AND Granted IS NOT NULL) OR
```

```
30          (Kind = 1 AND CardholderId IS NULL AND Granted IS NULL))
```

Line 20 `BIGINT`: events accumulate forever; an `INT` runs out at about two billion.

Line 21 Millisecond precision, no time zone: the column name says it's UTC.

Line 28 The rule from Figure 2.1 as SQL. A badge swipe (Kind 0) must have a cardholder and a decision; a status change (Kind 1) must have neither.

SQL snack2-access-event-log/src/AccessLog.Core/Data/schema.sql, lines 33–41

```
33  -- Both reports filter on a time window and then group by device. Keyed on time, so "the last
34  -- 24 hours" is a range seek; DeviceId and Granted are included so the reports never touch the
35  -- table itself.
36  IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'IX_AccessEvents_OccurredAt')
37  CREATE INDEX IX_AccessEvents_OccurredAt ON dbo.AccessEvents (OccurredAtUtc) INCLUDE (DeviceId, G
        ranted, Kind);
38
39  -- The event list filtered to one device, newest first.
40  IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'IX_AccessEvents_Device_OccurredAt')
41  CREATE INDEX IX_AccessEvents_Device_OccurredAt ON dbo.AccessEvents (DeviceId, OccurredAtUtc DESC
        );
```

Line 37 Keyed on time, so a time range is a seek. The `INCLUDE` columns ride along in the index's leaf pages without being part of its order: enough to answer both reports from the index alone.

Line 41 For the event list filtered to one device, newest first: the index is already in the order the query wants.

2.8 SQL Server with Dapper

C# snack2-access-event-log/src/AccessLog.Core/Data/SqlServerAccessEventRepository.cs, lines 23–47

```
23      /// <summary>Creates the database if it's missing, then the tables and indexes (schema.sql).
          </summary>
24      public static async Task EnsureDatabaseAsync(string connectionString)
25      {
26          var builder = new SqlConnectionStringBuilder(connectionString);
27          var database = builder.InitialCatalog;
28          if (string.IsNullOrEmpty(database))
29              throw new ArgumentException("The connection string needs a database name (Database=.
          ..).");
30
31          // CREATE DATABASE has to run from another database, so connect to master first.
32          builder.InitialCatalog = "master";
33          using (var master = new SqlConnection(builder.ConnectionString))
34          {
35              // EXEC() only takes strings and variables, so build the statement first. QUOTENAME
36              // brackets the name, so it can't be used to inject SQL.
37              await master.ExecuteAsync(
38                  """
39                  DECLARE @create NVARCHAR(300) = N'CREATE DATABASE ' + QUOTENAME(@database);
40                  IF DB_ID(@database) IS NULL EXEC (@create);
41                  """,
42                  new { database });
43          }
44
45          using var connection = new SqlConnection(connectionString);
46          await connection.ExecuteAsync(ReadSchema());
47      }
```

Line 32 You can't create a database while connected to it, so this connects to `master` first.

Line 39 `EXEC()` accepts only strings and variables, not a function call, so the statement is built first. (The first version passed `QUOTENAME(...)` straight to `EXEC`; CI's SQL Server job failed every database test until it was fixed.)

Line 39 Brackets the name and escapes any `]` in it, so a database name can't smuggle SQL in. The name itself arrives as a parameter, never pasted into the string.

C# snack2-access-event-log/src/AccessLog.Core/Data/SqlServerAccessEventRepository.cs, lines 99–130

```
 99      public async Task<long> RecordAsync(AccessEvent accessEvent)
100      {
101          using var connection = Open();
102          try
103          {
104              accessEvent.Id = await connection.ExecuteScalarAsync<long>(
105                  """
106                  INSERT INTO dbo.AccessEvents (OccurredAtUtc, DeviceId, CardholderId, Kind, Grant
    ed, Detail)
107                  OUTPUT INSERTED.Id
108                  VALUES (@OccurredAtUtc, @DeviceId, @CardholderId, @Kind, @Granted, @Detail);
109                  """,
110                  new
111                  {
112                      OccurredAtUtc = Utc(accessEvent.OccurredAtUtc),
113                      accessEvent.DeviceId,
114                      accessEvent.CardholderId,
115                      Kind = (byte)accessEvent.Kind,
116                      accessEvent.Granted,
117                      accessEvent.Detail,
118                  });
119          }
120          // SQL Server uses error 547 for every constraint conflict, foreign key or CHECK; the me
    ssage says which.
121          catch (SqlException e) when (e.Number == ConstraintViolation && e.Message.Contains("FORE
    IGN KEY"))
122          {
123              throw new UnknownReferenceException($"No device {accessEvent.DeviceId} or cardholder
     {accessEvent.CardholderId}.");
124          }
125          catch (SqlException e) when (e.Number == ConstraintViolation && e.Message.Contains("CK_A
    ccessEvents_Shape"))
126          {
127              throw new InvalidOperationException(InMemoryAccessEventRepository.ShapeRule, e);
128          }
129          return accessEvent.Id;
130      }
```

Line 65 Returns the new identity value from the same statement: no second round trip, and no `SCOPE_IDENTITY()` pitfalls.

Line 115 The column is `TINYINT`; passing a `byte` makes the parameter's type match it exactly.

Line 121 An exception **filter**: only this error is caught and translated; anything else propagates untouched, stack trace intact. SQL Server uses the same number, 547, for every constraint conflict, so the filter also checks which kind. The first version translated every 547 into "unknown device", and the contract test for the CHECK rule caught it on CI's SQL Server.

Line 125 The CHECK constraint becomes the same exception, with the same message, as the in-memory store's rule.

C# snack2-access-event-log/src/AccessLog.Core/Data/SqlServerAccessEventRepository.cs, lines 132–166

```
132     public async Task<IReadOnlyList<AccessEventView>> SearchAsync(EventQuery query)
133     {
134         // Each filter is "parameter is null OR condition", so one statement serves every combin
        ation.
135         // OPTION (RECOMPILE) lets SQL Server plan for the filters actually given each time.
136         const string sql = """
137             SELECT TOP (@Limit)
138                    e.Id, e.OccurredAtUtc, e.DeviceId, d.Name AS DeviceName, d.Location,
139                    e.CardholderId, c.Name AS CardholderName, e.Kind, e.Granted, e.Detail
140               FROM dbo.AccessEvents e
141               JOIN dbo.Devices d          ON d.Id = e.DeviceId
142               LEFT JOIN dbo.Cardholders c ON c.Id = e.CardholderId
143              WHERE (@FromUtc IS NULL      OR e.OccurredAtUtc >= @FromUtc)
144                AND (@ToUtc IS NULL        OR e.OccurredAtUtc <  @ToUtc)
145                AND (@DeviceId IS NULL     OR e.DeviceId = @DeviceId)
146                AND (@CardholderId IS NULL OR e.CardholderId = @CardholderId)
147                AND (@Kind IS NULL         OR e.Kind = @Kind)
148                AND (@Granted IS NULL      OR e.Granted = @Granted)
149                AND (@Text IS NULL         OR d.Name LIKE @Text OR d.Location LIKE @Text OR c.Nam
        e LIKE @Text)
150              ORDER BY e.OccurredAtUtc DESC, e.Id DESC
151             OPTION (RECOMPILE);
152             """;
153         using var connection = Open();
154         var rows = await connection.QueryAsync<AccessEventView>(sql, new
155         {
156             Limit = Math.Max(0, query.Limit),
157             FromUtc = query.FromUtc.HasValue ? Utc(query.FromUtc.Value) : (DateTime?)null,
158             ToUtc = query.ToUtc.HasValue ? Utc(query.ToUtc.Value) : (DateTime?)null,
159             query.DeviceId,
160             query.CardholderId,
161             Kind = query.Kind.HasValue ? (byte)query.Kind.Value : (byte?)null,
162             query.Granted,
163             Text = string.IsNullOrWhiteSpace(query.Text) ? null : "%" + EscapeLike(query.Text!.T
        rim()) + "%",
164         });
165         return rows.Select(AsUtc).ToList();
166     }
```

Line 143 The "optional filter" pattern: when a parameter is null the condition is always true. One statement serves every combination of filters.

Line 135 Without it, SQL Server caches one plan for this statement, made for whichever filters the first call used, and reuses it for very different ones. Recompiling costs a little CPU per call and gets a plan for the actual values.

Line 163 The pattern is built in C# and sent as a parameter, still never pasted into the SQL.

Line 165 `DATETIME2` has no time zone, so values come back as `DateTimeKind.Unspecified`; they're marked UTC before anyone can mistake them for local times.

C# snack2-access-event-log/src/AccessLog.Core/Data/SqlServerAccessEventRepository.cs, lines 168–185

```
168     public async Task<IReadOnlyList<DeniedByDoorRow>> DeniedByDoorAsync(DateTime sinceUtc)
169     {
170         // GROUP BY the door; only denied swipes in the window count. The time-keyed index makes
        the
171         // WHERE a range seek, and it includes DeviceId and Granted, so the table itself isn't r
        ead.
172         const string sql = """
173             SELECT d.Id AS DeviceId, d.Name AS DeviceName, d.Location, COUNT(*) AS Denied
174               FROM dbo.AccessEvents e
175               JOIN dbo.Devices d ON d.Id = e.DeviceId
```

```
             WHERE e.OccurredAtUtc >= @sinceUtc
               AND e.Kind = 0
               AND e.Granted = 0
             GROUP BY d.Id, d.Name, d.Location
             ORDER BY Denied DESC, d.Name;
            """;
        using var connection = Open();
        var rows = await connection.QueryAsync<DeniedByDoorRow>(sql, new { sinceUtc = Utc(sinceU
      tc) });
        return rows.ToList();
    }
```

The first report, as SQL you could paste into SQL Server Management Studio. `WHERE` narrows to denied badge swipes in the window (a range seek on the time index); `GROUP BY` makes one row per door; `COUNT(*)` counts each group. Every non-aggregated column in the `SELECT` is in the `GROUP BY`, which SQL requires. The ordering tie-break on name makes the result deterministic.

C# snack2-access-event-log/src/AccessLog.Core/Data/SqlServerAccessEventRepository.cs, lines 187–200

```
    public async Task<IReadOnlyList<NoisyDeviceRow>> NoisiestDevicesAsync(DateTime sinceUtc, int
       top)
    {
        const string sql = """
            SELECT TOP (@top) d.Id AS DeviceId, d.Name AS DeviceName, d.Kind, COUNT(*) AS Events
              FROM dbo.AccessEvents e
              JOIN dbo.Devices d ON d.Id = e.DeviceId
             WHERE e.OccurredAtUtc >= @sinceUtc
             GROUP BY d.Id, d.Name, d.Kind
             ORDER BY Events DESC, d.Name;
            """;
        using var connection = Open();
        var rows = await connection.QueryAsync<NoisyDeviceRow>(sql, new { sinceUtc = Utc(sinceUt
      c), top = Math.Max(0, top) });
        return rows.ToList();
    }
```

`TOP (@top)` with parentheses accepts a parameter. Every kind of event counts here, so a flapping camera shows up as noisy, which is the point of the report.

C# snack2-access-event-log/src/AccessLog.Core/Data/SqlServerAccessEventRepository.cs, lines 212–214

```
    /// <summary>Makes %, _ and [ in user text match themselves in a LIKE pattern.</summary>
    private static string EscapeLike(string text) =>
        text.Replace("[", "[[]").Replace("%", "[%]").Replace("_", "[_]");
```

In a `LIKE` pattern, `%` and _ are wildcards. Wrapping them in brackets makes them literal, so searching for "_" finds underscores instead of everything. A contract test checks it.

2.9 The in-memory twin

C# snack2-access-event-log/src/AccessLog.Core/Data/InMemoryAccessEventRepository.cs, lines 55–77

```
55      public Task<long> RecordAsync(AccessEvent accessEvent)
56      {
57          lock (_gate)
58          {
59              if (_devices.All(d => d.Id != accessEvent.DeviceId) ||
60                  (accessEvent.CardholderId is int c && _cardholders.All(x => x.Id != c)))
61                  throw new UnknownReferenceException($"No device {accessEvent.DeviceId} or cardho
    lder {accessEvent.CardholderId}.");
62              CheckShape(accessEvent);
63
64              accessEvent.Id = _nextEventId++;
65              _events.Add(new AccessEvent
66              {
67                  Id = accessEvent.Id,
68                  OccurredAtUtc = Utc(accessEvent.OccurredAtUtc),
69                  DeviceId = accessEvent.DeviceId,
70                  CardholderId = accessEvent.CardholderId,
71                  Kind = accessEvent.Kind,
72                  Granted = accessEvent.Granted,
73                  Detail = accessEvent.Detail,
74              });
75              return Task.FromResult(accessEvent.Id);
76          }
77      }
```

Line 61 The foreign keys, in code: same exception as SQL Server's translation.

Line 62 The `CHECK` constraint, in code.

Line 68 Stored the way SQL Server would store it: UTC, to the millisecond (see `Utc` at the bottom of the file).

C# snack2-access-event-log/src/AccessLog.Core/Data/InMemoryAccessEventRepository.cs, lines 79–97

```
79      public Task<IReadOnlyList<AccessEventView>> SearchAsync(EventQuery query)
80      {
81          lock (_gate)
82          {
83              var text = string.IsNullOrWhiteSpace(query.Text) ? null : query.Text!.Trim();
84              var rows = Views()
85                  .Where(e => query.FromUtc is not DateTime start || e.OccurredAtUtc >= Utc(start)
    )
86                  .Where(e => query.ToUtc is not DateTime end || e.OccurredAtUtc < Utc(end))
87                  .Where(e => query.DeviceId is null || e.DeviceId == query.DeviceId)
88                  .Where(e => query.CardholderId is null || e.CardholderId == query.CardholderId)
89                  .Where(e => query.Kind is null || e.Kind == query.Kind)
90                  .Where(e => query.Granted is null || e.Granted == query.Granted)
91                  .Where(e => text is null || Contains(e.DeviceName, text) || Contains(e.Location,
     text) || Contains(e.CardholderName, text))
92                  .OrderByDescending(e => e.OccurredAtUtc).ThenByDescending(e => e.Id)
93                  .Take(Math.Max(0, query.Limit))
94                  .ToList();
95              return Task.FromResult<IReadOnlyList<AccessEventView>>(rows);
96          }
97      }
```

The same filters as the SQL, in LINQ, with the same ordering (time, then id, both descending). The text match uses `IndexOf(..., OrdinalIgnoreCase)` because .NET Framework has no case-insensitive `Contains`; SQL Server's default collation is case-insensitive too.

C# snack2-access-event-log/src/AccessLog.Core/Data/InMemoryAccessEventRepository.cs, lines 145–162

```
145     private IEnumerable<AccessEventView> Views() =>
146         from e in _events
147         join d in _devices on e.DeviceId equals d.Id
148         join c in _cardholders on e.CardholderId equals (int?)c.Id into cs
149         from c in cs.DefaultIfEmpty()
150         select new AccessEventView
151         {
152             Id = e.Id,
153             OccurredAtUtc = e.OccurredAtUtc,
154             DeviceId = d.Id,
155             DeviceName = d.Name,
156             Location = d.Location,
157             CardholderId = e.CardholderId,
158             CardholderName = c?.Name,
159             Kind = e.Kind,
160             Granted = e.Granted,
161             Detail = e.Detail,
162         };
```

A LINQ **left join**: `join ... into cs` groups the matching cardholders, and `DefaultIfEmpty()` yields a single null when there are none, so status changes survive the join. The `(int?)c.Id` cast is needed because `CardholderId` is nullable and the two sides of `equals` must have the same type.

2.10 The simulator

C# snack2-access-event-log/src/AccessLog.Core/Simulation/AccessSimulator.cs, lines 10–14

```
10  /// <summary>The current time, injectable so tests can fix it.</summary>
11  public interface IClock
12  {
13      DateTime UtcNow { get; }
14  }
```

The current time behind an interface, so tests can freeze it. Code that calls `DateTime.UtcNow` directly can't be tested at "exactly 24 hours ago".

C# snack2-access-event-log/src/AccessLog.Core/Simulation/AccessSimulator.cs, lines 73–88

```
73      private AccessEvent Swipe(DateTime atUtc)
74      {
75          // Some doors are busier than others: pick from the first doors more often.
76          var door = _doors[Math.Min(random.Next(_doors.Count) , random.Next(_doors.Count))];
77          var cardholder = _cardholders[random.Next(_cardholders.Count)];
78          var granted = !RevokedBadges.Contains(cardholder.BadgeNumber) && random.Next(100) < 92;
79          return new AccessEvent
80          {
81              OccurredAtUtc = atUtc,
82              DeviceId = door.Id,
83              CardholderId = cardholder.Id,
84              Kind = EventKind.BadgeSwipe,
85              Granted = granted,
86              Detail = granted ? null : "Badge refused",
87          };
88      }
```

Line 76 The smaller of two random picks favours the first doors, so the noisiest-device report has a clear winner instead of a tie.

Line 78 Two badges are always refused, so the denied report always has something to show; a test checks they're never granted.

C# snack2-access-event-log/src/AccessLog.Core/Simulation/AccessSimulator.cs, lines 90–104

```
90      private AccessEvent StatusChange(DateTime atUtc)
91      {
92          var camera = _cameras[random.Next(_cameras.Count)];
93          var from = _cameraStatus.TryGetValue(camera.Id, out var current) ? current : "Online";
94          var options = new[] { "Online", "Offline", "Alarm" }.Where(s => s != from).ToArray();
95          var to = options[random.Next(options.Length)];
96          _cameraStatus[camera.Id] = to;
97          return new AccessEvent
98          {
99              OccurredAtUtc = atUtc,
100             DeviceId = camera.Id,
101             Kind = EventKind.StatusChange,
102             Detail = $"{from} → {to}",
103         };
104     }
```

Each camera remembers its last status, so the history reads like Snack 1's: "Online → Alarm", then "Alarm → Offline", never two "from" values that don't follow on. A test walks every camera's history to check it.

2.11 The ViewModel

C# snack2-access-event-log/src/AccessLog.Core/Mvvm/Mvvm.cs, lines 27–56

```
27  /// <summary>
28  /// A command that runs async work. It disables itself while running, so a double-click can't
29  /// start the same database query twice. <see cref="ExecuteAsync"/> lets tests await the work.
30  /// </summary>
31  public sealed class AsyncRelayCommand(Func<Task> execute) : ICommand
32  {
33      private bool _running;
34
35      public event EventHandler? CanExecuteChanged;
36
37      public bool CanExecute(object? parameter) => !_running;
38
39      public async void Execute(object? parameter) => await ExecuteAsync();
40
41      public async Task ExecuteAsync()
42      {
43          if (_running) return;
44          _running = true;
45          CanExecuteChanged?.Invoke(this, EventArgs.Empty);
46          try
47          {
48              await execute();
49          }
50          finally
51          {
52              _running = false;
53              CanExecuteChanged?.Invoke(this, EventArgs.Empty);
54          }
55      }
56  }
```

Line 37 The button disables itself while its query runs, so a double-click can't send the same query twice.

Line 39 `async void` is only acceptable for event-handler-like entry points such as `ICommand.Execute`, which must return void. Tests call `ExecuteAsync` and await it.

C# snack2-access-event-log/src/AccessLog.Core/ViewModels/EventLogViewModel.cs, lines 101–117

```
101     /// <summary>The query the current filters describe.</summary>
102     public EventQuery CurrentQuery() => new()
103     {
104         Text = string.IsNullOrWhiteSpace(SearchText) ? null : SearchText.Trim(),
105         Kind = Outcome switch
106         {
107             OutcomeFilter.Granted or OutcomeFilter.Denied => EventKind.BadgeSwipe,
108             OutcomeFilter.StatusChanges => EventKind.StatusChange,
109             _ => null,
110         },
111         Granted = Outcome switch
112         {
113             OutcomeFilter.Granted => true,
114             OutcomeFilter.Denied => false,
115             _ => null,
116         },
117         Limit = 200,
```

The filters as a query object, separate from running it, so a test can check the translation without a database: "Denied" means badge swipes with `Granted = false`.

C# snack2-access-event-log/src/AccessLog.Core/ViewModels/EventLogViewModel.cs, lines 120–133

```
120     /// <summary>
121     /// Reloads the list and both reports. If a refresh is already running, this one waits for i
        t
122     /// and then runs once more, so the last filter change always wins and queries never overlap
        .
123     /// </summary>
124     public Task RefreshAsync()
125     {
126         if (_refreshing is { IsCompleted: false })
127         {
128             _refreshAgain = true;
129             return _refreshing;
130         }
131         _refreshing = RefreshLoopAsync();
132         return _refreshing;
133     }
```

C# snack2-access-event-log/src/AccessLog.Core/ViewModels/EventLogViewModel.cs, lines 135–158

```
135     private async Task RefreshLoopAsync()
136     {
137         do
138         {
139             _refreshAgain = false;
140             try
141             {
142                 var since = _clock.UtcNow.AddHours(-ReportHours);
143                 var events = await _repository.SearchAsync(CurrentQuery());
144                 var denied = await _repository.DeniedByDoorAsync(since);
145                 var noisy = await _repository.NoisiestDevicesAsync(since, TopDevices);
146 
147                 Replace(Events, events);
148                 Replace(DeniedByDoor, denied);
149                 Replace(NoisiestDevices, noisy);
150                 LastRefreshedUtc = _clock.UtcNow;
151                 Status = $"{events.Count} events shown · {denied.Sum(d => d.Denied)} denied in t
          he last {ReportHours} h";
```

```
            }
            catch (Exception e)
            {
                Status = "Couldn't load events: " + e.Message;
            }
        } while (_refreshAgain);
    }
```

Line 126 Typing "server" sets the search five times in a row. Rather than run five overlapping queries (whose answers could arrive out of order, leaving the list showing an older filter), a refresh that arrives mid-query just asks for one more pass.

Line 157 The loop runs again if anything asked while it was busy, so the last filter always wins and queries never overlap.

Line 155 A database outage becomes a message in the status bar, not a crashed app. A test with a broken repository checks it.

2.12 The .NET Framework 4.8 app

MSBuild snack2-access-event-log/src/AccessLog.App/AccessLog.App.csproj, lines 1–19

```
<Project Sdk="Microsoft.NET.Sdk">

  <!-- The WPF app on .NET Framework 4.8, as the README asks. Everything it shows comes from Acc
      essLog.Core. -->
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net48</TargetFramework>
    <UseWPF>true</UseWPF>
    <AssemblyName>AccessLog</AssemblyName>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.NETFramework.ReferenceAssemblies" Version="1.0.3" Priva
      teAssets="all" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\AccessLog.Core\AccessLog.Core.csproj" />
  </ItemGroup>

</Project>
```

An SDK-style project targeting `net48`: the modern project format, built by the .NET 10 SDK, producing an app for .NET Framework 4.8. No `ImplicitUsings` here: on .NET Framework they include `System.Net.Http`, which isn't referenced by default, and the build fails.

C# snack2-access-event-log/src/AccessLog.App/Services/ViewModelLocator.cs, lines 24–48

```
    private static EventLogViewModel Create()
    {
        var clock = new SystemClock();
        // Startup work runs on a pool thread and is waited for here, so the async database
        // calls never need the UI thread (waiting on it from the UI thread would deadlock).
        var (repository, description) = Task.Run(() => ChooseStoreAsync()).GetAwaiter().GetResul
      t();
        var simulator = new AccessSimulator(repository, new Random(), clock);
        Task.Run(async () =>
        {
            await simulator.SeedAsync();
            if ((await repository.SearchAsync(new Core.Model.EventQuery { Limit = 1 })).Count ==
```

```
            0)
35                  await simulator.BackfillAsync(hours: 24, events: 400);
36          }).GetAwaiter().GetResult();
37
38          var viewModel = new EventLogViewModel(repository, clock, description);
39          var timer = new DispatcherTimer { Interval = TimeSpan.FromSeconds(1.5) };
40          timer.Tick += async (_, _) =>
41          {
42              await simulator.StepAsync();
43              await viewModel.RefreshAsync();
44          };
45          timer.Start();
46          _ = viewModel.RefreshAsync();
47          return viewModel;
48      }
```

Line 29 Waiting on async code from the UI thread deadlocks if the async code needs the UI thread to continue (it captures the UI's synchronization context). `Task.Run` moves the work to a pool thread, which has no such context, so waiting is safe. It's acceptable once, at startup; never in an event handler.

Line 39 A `DispatcherTimer` ticks on the UI thread, so its handler can update bound collections directly; no dispatcher hop needed, unlike Snack 1's thread-pool timer.

C# snack2-access-event-log/src/AccessLog.App/Services/ViewModelLocator.cs, lines 50–64

```
50      /// <summary>SQL Server if it answers within a few seconds, otherwise memory (and say so).</
          summary>
51      private static async Task<(IAccessEventRepository, string)> ChooseStoreAsync()
52      {
53          var connection = Environment.GetEnvironmentVariable("ACCESSLOG_CONNECTION") ?? DefaultCo
          nnection;
54          try
55          {
56              await SqlServerAccessEventRepository.EnsureDatabaseAsync(connection);
57              var builder = new Microsoft.Data.SqlClient.SqlConnectionStringBuilder(connection);
58              return (new SqlServerAccessEventRepository(connection), $"SQL Server: {builder.Initi
          alCatalog} on {builder.DataSource}");
59          }
60          catch (Exception e)
61          {
62              return (new InMemoryAccessEventRepository(), $"In memory (SQL Server not reachable:
          {e.Message})");
63          }
64      }
```

SQL Server if it answers (the connection string has `Connect Timeout=3`), otherwise memory, and the status bar says which and why. A demo that starts on any machine is worth more than one that crashes without Docker.

XAML snack2-access-event-log/src/AccessLog.App/MainWindow.xaml, lines 56–89

```
56              <!-- Reports -->
57              <TabItem Header="Reports">
58                  <Grid Margin="8">
59                      <Grid.ColumnDefinitions>
60                          <ColumnDefinition Width="*" />
61                          <ColumnDefinition Width="16" />
62                          <ColumnDefinition Width="*" />
63                      </Grid.ColumnDefinitions>
64                      <DockPanel>
65                          <TextBlock DockPanel.Dock="Top" Text="Denied access by door, last 24 h"
66                                     FontWeight="SemiBold" Margin="0,0,0,8" />
67                          <DataGrid ItemsSource="{Binding DeniedByDoor}" AutoGenerateColumns="Fals
```

```
e" IsReadOnly="True"
                                  HeadersVisibility="Column">
                          <DataGrid.Columns>
                              <DataGridTextColumn Header="Door" Width="*" Binding="{Binding De
viceName}" />
                              <DataGridTextColumn Header="Location" Width="*" Binding="{Bindin
g Location}" />
                              <DataGridTextColumn Header="Denied" Width="80" Binding="{Binding
 Denied}" />
                          </DataGrid.Columns>
                      </DataGrid>
                  </DockPanel>
                  <DockPanel Grid.Column="2">
                      <TextBlock DockPanel.Dock="Top" Text="Top 5 noisiest devices, last 24 h"
                                 FontWeight="SemiBold" Margin="0,0,0,8" />
                      <DataGrid ItemsSource="{Binding NoisiestDevices}" AutoGenerateColumns="F
alse" IsReadOnly="True"
                                HeadersVisibility="Column">
                          <DataGrid.Columns>
                              <DataGridTextColumn Header="Device" Width="*" Binding="{Binding
DeviceName}" />
                              <DataGridTextColumn Header="Kind" Width="90" Binding="{Binding K
ind}" />
                              <DataGridTextColumn Header="Events" Width="80" Binding="{Binding
 Events}" />
                          </DataGrid.Columns>
                      </DataGrid>
                  </DockPanel>
              </Grid>
          </TabItem>
```

Two `DataGrid`s side by side, each bound to one report collection. `AutoGenerateColumns="False"` with explicit columns keeps the headers human and the order fixed.

2.13 The contract tests

C# snack2-access-event-log/tests/AccessLog.Tests/RepositoryContract.cs, lines 6–39

```
/// <summary>
/// The behaviour every IAccessEventRepository must have. InMemoryRepositoryTests and
/// SqlServerRepositoryTests inherit these tests, so both stores are held to exactly the same ru
    les,
/// and the in-memory one can stand in for SQL Server everywhere else.
/// </summary>
public abstract class RepositoryContract : IAsyncLifetime
{
    // Whole seconds: SQL Server keeps milliseconds (DATETIME2(3)), so finer times wouldn't roun
      d-trip.
    protected static readonly DateTime Now = new(2026, 11, 12, 15, 0, 0, DateTimeKind.Utc);

    protected IAccessEventRepository Repo { get; private set; } = null!;
    protected Device MainDoor { get; private set; } = null!;
    protected Device ServerRoom { get; private set; } = null!;
    protected Device Lobby { get; private set; } = null!;
    protected Cardholder Alice { get; private set; } = null!;
    protected Cardholder Bob { get; private set; } = null!;

    /// <summary>A new, empty store for each test.</summary>
    protected abstract Task<IAccessEventRepository> CreateAsync();

    public virtual async ValueTask InitializeAsync()
    {
        Repo = await CreateAsync();
        MainDoor = new Device { Name = "Main entrance", Kind = DeviceKind.Door, Location = "Grou
      nd floor" };
        ServerRoom = new Device { Name = "Server room", Kind = DeviceKind.Door, Location = "Floo
      r 2" };
```

```
31          Lobby = new Device { Name = "Lobby camera", Kind = DeviceKind.Camera, Location = "Ground
            floor" };
32          foreach (var d in new[] { MainDoor, ServerRoom, Lobby }) await Repo.AddDeviceAsync(d);
33          Alice = new Cardholder { Name = "Alice Martin", BadgeNumber = "B-1" };
34          Bob = new Cardholder { Name = "Bob Tremblay", BadgeNumber = "B-2" };
35          await Repo.AddCardholderAsync(Alice);
36          await Repo.AddCardholderAsync(Bob);
37      }
38
39      public virtual ValueTask DisposeAsync() => ValueTask.CompletedTask;
```

Line 14 Whole seconds on a fixed date: SQL Server keeps milliseconds, so finer times wouldn't come back equal, and a fixed date makes failures reproducible.

Line 24 The only thing a subclass provides: a fresh, empty store. Every `[Fact]` below runs once per subclass.

C# snack2-access-event-log/tests/AccessLog.Tests/RepositoryContract.cs, lines 192–205

```
192     public async Task Denied_by_door_counts_only_denied_swipes_in_the_window_most_first()
193     {
194         await Swipe(ServerRoom, Alice, false, Now.AddHours(-1));
195         await Swipe(ServerRoom, Bob, false, Now.AddHours(-2));
196         await Swipe(MainDoor, Bob, false, Now.AddHours(-3));
197         await Swipe(MainDoor, Alice, true, Now.AddHours(-1));          // granted: doesn't count
198         await Swipe(MainDoor, Alice, false, Now.AddHours(-25));        // too old: doesn't count
199         await StatusChange(Lobby, "Online → Alarm", Now.AddHours(-1)); // not a swipe
200
201         var rows = await Repo.DeniedByDoorAsync(Now.AddHours(-24));
202
203         Assert.Equal(new[] { ("Server room", 2), ("Main entrance", 1) }, rows.Select(r => (r.Dev
            iceName, r.Denied)));
204         Assert.Equal("Floor 2", rows[0].Location);
205     }
```

Each line of setup is there to be excluded for a reason the comment gives: granted, too old, not a swipe. If the SQL's `WHERE` or the LINQ's `Where` forgets one condition, this test fails for that store.

C# snack2-access-event-log/tests/AccessLog.Tests/SqlServerRepositoryTests.cs, lines 20–24

```
20      public override ValueTask InitializeAsync()
21      {
22          Assert.SkipWhen(Server is null, "Set TEST_SQLSERVER to a SQL Server connection string (n
            o database name) to run these.");
23          return base.InitializeAsync();
24      }
```

C# snack2-access-event-log/tests/AccessLog.Tests/SqlServerRepositoryTests.cs, lines 54–80

```
54      public async Task The_reports_are_answered_from_the_time_index()
55      {
56          // On a nearly empty table, reading everything is cheapest and SQL Server rightly ignore
            s
57          // indexes. So first give it a realistic table: 20,000 events over 60 days.
58          await using var connection = new SqlConnection(ConnectionString);
59          await connection.OpenAsync(TestContext.Current.CancellationToken);
60          await connection.ExecuteAsync("""
61              INSERT INTO dbo.AccessEvents (OccurredAtUtc, DeviceId, CardholderId, Kind, Granted)
62              SELECT TOP (20000)
63                     DATEADD(MINUTE, -ABS(CHECKSUM(NEWID())) % (60 * 24 * 60), SYSUTCDATETIME()),
64                     @door, @alice, 0, CASE WHEN ABS(CHECKSUM(NEWID())) % 10 = 0 THEN 0 ELSE 1 END
65                FROM sys.all_objects a CROSS JOIN sys.all_objects b;
66              UPDATE STATISTICS dbo.AccessEvents;
67              """, new { door = ServerRoom.Id, alice = Alice.Id });
```

```
68
69          // Ask for the estimated plan of the "denied by door" query instead of running it, and c
          heck
70          // it uses the index the schema creates for exactly this purpose.
71          await connection.ExecuteAsync("SET SHOWPLAN_XML ON;");
72          var plan = await connection.ExecuteScalarAsync<string>("""
73              SELECT d.Id, d.Name, d.Location, COUNT(*) FROM dbo.AccessEvents e JOIN dbo.Devices d
           ON d.Id = e.DeviceId
74                WHERE e.OccurredAtUtc >= DATEADD(HOUR, -24, SYSUTCDATETIME()) AND e.Kind = 0 AND e.
          Granted = 0
75                GROUP BY d.Id, d.Name, d.Location;
76              """);
77          await connection.ExecuteAsync("SET SHOWPLAN_XML OFF;");
78
79          Assert.Contains("IX_AccessEvents_OccurredAt", plan);
80      }
```

A test of a performance decision. `SET SHOWPLAN_XML ON` makes SQL Server return its execution plan instead of running the query; the test checks the plan names the index. If someone drops the index or rewrites the report so it can't use it, this fails. The 20,000 rows matter: the first version of this test ran on a table of a dozen rows, where reading the whole table *is* the cheapest plan, and SQL Server rightly ignored the index. Testing an optimizer decision needs data shaped like production.

Class	Runs	What it adds
InMemoryRepositoryTests	20 contract tests	the contract on the in-memory store
SqlServerRepositoryTests	20 contract tests + 2	the schema re-runs cleanly; the plan uses the index
EventLogViewModelTests	9	refresh, filters, clear, overlapping refreshes, a failing store
AccessSimulatorTests	6	seeding once, valid events, revoked badges, status sequences, backfill

2.14 Interview questions

Why put a CHECK constraint in the database when the code already validates?

The database is shared by every client and every future version of this one. A constraint makes invalid rows impossible regardless of who writes them; validation in code only protects the code that runs it.

What's a covering index, and why does the report need one?

An index containing every column a query uses, either as keys or as INCLUDE columns, so the query is answered from the index without reading the table. The report filters on time and needs DeviceId, Kind and Granted; keying on time makes the window a seek, and including the rest makes it covering.

What does OPTION (RECOMPILE) fix in the search query?

Parameter sniffing: SQL Server caches one plan per statement, made for the first parameter values it saw. With "optional filter" predicates, a plan made for "no filters" is terrible for "one device", and vice versa. Recompiling makes a plan for the actual values each time.

How do you stop SQL injection with a search box?

Never build SQL from input: the text goes in as a parameter. The LIKE pattern is assembled in C# (with % and _ escaped) and passed as a parameter too. The database name in CREATE DATABASE is passed as a parameter and bracketed with QUOTENAME.

Why store UTC?

Local time is ambiguous (the hour repeated when clocks go back) and differs between sites. UTC is one timeline; convert to local only for display.

Why Dapper and not Entity Framework here?

The reports are the deliverable, so readable SQL matters, and EF Core doesn't support .NET Framework. Dapper does the tedious part (parameters, mapping) and nothing else.

How do you know the in-memory repository behaves like SQL Server?

The same contract tests run against both. Any difference in ordering, filtering or reporting fails a test for one of them.

2.15 Exercises

Paging

Add "load more" to the event list without OFFSET scanning millions of rows.

Hint. Keyset paging: pass the last row's (OccurredAtUtc, Id) and ask for rows before it; the existing ordering and indexes make that a seek. Add it to the contract.

Retention

Delete events older than 90 days without blocking the app.

Hint. Delete in batches of a few thousand in a loop (`DELETE TOP (5000) ... WHERE OccurredAtUtc < @cutoff`); the time index makes each batch cheap.

Denied by hour

A third report: denials per hour of the day over the last week.

Hint. `DATEPART(HOUR, OccurredAtUtc)` in the GROUP BY; decide whether the hours are UTC or local and say so in the UI.

Migrations

Add a column (the door's zone) to a database that already has data.

Hint. Write schema changes as numbered scripts and record which ran in a table; or adopt a tool such as DbUp.

Part 3

Mini Plugin Framework

A host that knows one interface and nothing else, modules found in folders at run time, each in its own assembly load context, and an event router that keeps one broken module from taking down the rest. This is the "framework so each project doesn't start from scratch" idea from Genetec's job ad, in miniature.

3.1 What it does

`solution-host` scans a modules folder, loads every module it finds, and plays a simulated evening at a building through them: badge swipes, doors unlocked, held open and forced. Two sample modules react. **DoorForcedEmail** "emails" the security desk (it logs the email) when a door is forced, at most once per door every five minutes. **AfterHoursAutoLock** asks the host to lock any door left unlocked or held open between 19:00 and 07:00. The host carries out the lock and reports a `DoorLocked` event back to every module.

Neither the host nor the other module knows these modules exist. Adding a third means building a DLL and copying its folder.

3.2 Plugins, reflection and load contexts

Reflection lets a program inspect types at run time: load an assembly from a path, list its public types, ask whether one implements an interface, and create it. That's all a plugin host needs to find code it was never compiled against.

Assembly load contexts (ALCs) are .NET's way of keeping plugins apart. Every assembly lives in one; the app's own are in the default context. Loading each plugin into its own context means each plugin's dependencies are resolved from its own folder, so two plugins can use different versions of the same library, and a context can be unloaded.

There's one trap. A type's identity is its name *plus* the assembly instance it came from. If a plugin loaded its own copy of the contract assembly, its `ISolutionModule` would be a different type from the host's, and the host's cast would fail with a baffling `InvalidCastException` between two types with the same name. So the contract must exist exactly once, in the host's context, and every plugin must use that copy.

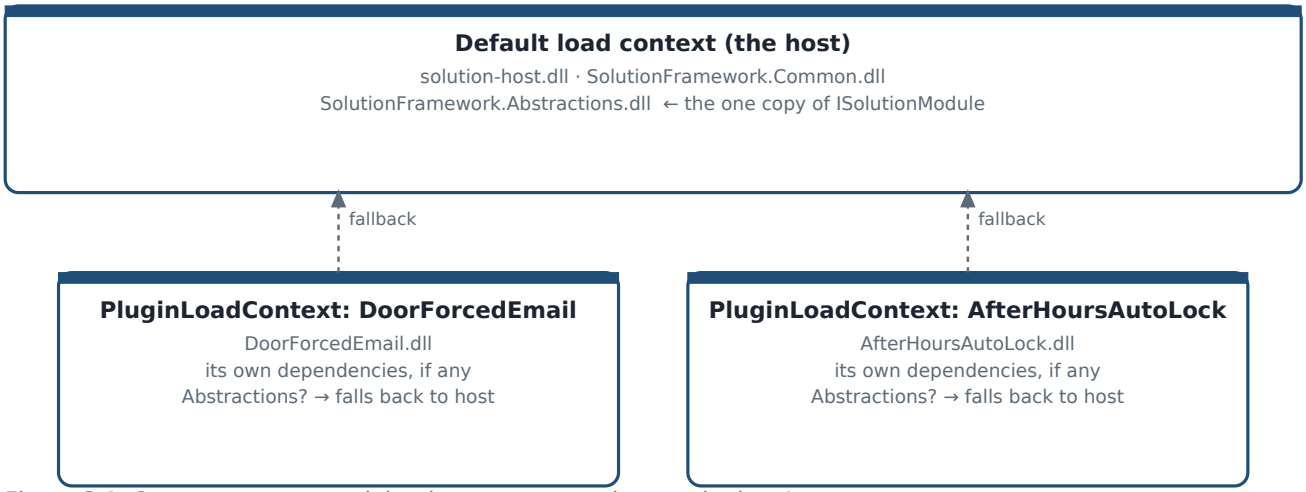


Figure 3.1. One context per module; the contract resolves to the host's copy.

3.3 The contract

MSBuild snack3-plugin-framework/src/SolutionFramework.Abstractions/SolutionFramework.Abstractions.csproj, lines 1–13

```
<Project Sdk="Microsoft.NET.Sdk">

  <!--
    The contract between the host and its modules, and nothing else. Modules compile against it
      but
    don't ship a copy: at run time every module uses the host's one, so ISolutionModule is the s
      ame
    type on both sides. Keep it small and stable; changing it is a breaking change for every mod
      ule.
  -->
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

</Project>
```

C# snack3-plugin-framework/src/SolutionFramework.Abstractions/Contracts.cs, lines 3–16

```
/// <summary>
/// A pluggable piece of a security solution. The host finds every public class that implements
/// this in the module folders, creates it, calls <see cref="Initialize"/> once, then passes it
      every
/// event. A module that throws doesn't stop the others.
/// </summary>
public interface ISolutionModule
{
    /// <summary>Also the name of the module's section in modules.json.</summary>
    string Name { get; }

    void Initialize(IModuleContext context);

    void OnEvent(SecurityEvent securityEvent);
}
```

Three members, as the README specifies. The doc comment is part of the contract: modules are created with a parameterless constructor, initialized once, then sent events.

C# snack3-plugin-framework/src/SolutionFramework.Abstractions/Contracts.cs, lines 18–47

```
/// <summary>What the host gives a module: its logger, its settings, a way to act, and the time.
      </summary>
public interface IModuleContext
{
    IModuleLogger Logger { get; }
    IModuleConfig Config { get; }
    ICommandSink Commands { get; }
    TimeProvider Time { get; }
}

public interface IModuleLogger
{
    void Info(string message);
    void Warn(string message);
    void Error(string message, Exception? exception = null);
}

/// <summary>A module's own settings (its section of modules.json).</summary>
public interface IModuleConfig
{
```

```
37      string? Get(string key);
38      T Get<T>(string key, T defaultValue) where T : IParsable<T>;
39  }
40
41  /// <summary>Modules ask the host to do things through commands; they never touch hardware thems
          elves.</summary>
42  public interface ICommandSink
43  {
44      void Send(DeviceCommand command);
45  }
46
47  public enum SecurityEventKind
```

Line 24 `TimeProvider` (.NET 8) is the framework's abstraction of the clock. Modules that ask it for the time can be tested at 18:59 and 19:00 without waiting.

Line 38 A generic setting reader constrained to types that can parse themselves (`int`, `double`, `TimeSpan`...), using **static abstract interface members**: `T.TryParse` is called on the type itself.

Line 42 Modules never touch hardware. They ask, and the host decides how and when.

C# snack3-plugin-framework/src/SolutionFramework.Abstractions/Contracts.cs, lines 47–61

```
47  public enum SecurityEventKind
48  {
49      AccessGranted,
50      AccessDenied,
51      DoorUnlocked,
52      DoorLocked,
53      DoorHeldOpen,
54      DoorForced,
55  }
56
57  public sealed record SecurityEvent(SecurityEventKind Kind, string DoorId, string DoorName, strin
          g Location, DateTimeOffset OccurredAt);
58
59  public abstract record DeviceCommand(string DoorId, string Reason);
60
61  public sealed record LockDoorCommand(string DoorId, string Reason) : DeviceCommand(DoorId, Reaso
          n);
```

Events and commands are records: immutable, compared by value, with a readable `ToString`. `DeviceCommand` is an abstract record so the host can accept any command and switch on its type.

3.4 Shared logging and configuration

C# snack3-plugin-framework/src/SolutionFramework.Common/Logging.cs, lines 12–16

```
12  /// <summary>Where log entries go. The host writes to the console; tests keep them in memory.</s
          ummary>
13  public interface ILogSink
14  {
15      void Write(LogEntry entry);
16  }
```

C# snack3-plugin-framework/src/SolutionFramework.Common/Logging.cs, lines 47–56

```
47  /// <summary>A logger that stamps every entry with its source (the module name) and the time.</s
      ummary>
48  public sealed class SourceLogger(string source, ILogSink sink, TimeProvider time) : IModuleLogge
      r
49  {
50      public void Info(string message) => Write(LogLevel.Info, message, null);
51      public void Warn(string message) => Write(LogLevel.Warn, message, null);
52      public void Error(string message, Exception? exception = null) => Write(LogLevel.Error, mess
        age, exception);
53
54      private void Write(LogLevel level, string message, Exception? exception) =>
55          sink.Write(new LogEntry(time.GetLocalNow(), level, source, message, exception));
56  }
```

Every module gets a logger stamped with its own name, so the console reads `21:42:00 WARN [DoorForcedEmail] EMAIL to ...`. Where log entries go (console, memory for tests) is a sink chosen by the host. The time comes from the same `TimeProvider` as everything else, so log lines show simulated time.

C# snack3-plugin-framework/src/SolutionFramework.Common/Configuration.cs, lines 22–35

```
22      public static ModuleSettings Parse(string json)
23      {
24          using var document = JsonDocument.Parse(json, new JsonDocumentOptions { CommentHandling
          = JsonCommentHandling.Skip, AllowTrailingCommas = true });
25          var sections = new Dictionary<string, Dictionary<string, string>>(StringComparer.Ordinal
        IgnoreCase);
26          foreach (var section in document.RootElement.EnumerateObject())
27          {
28              if (section.Value.ValueKind != JsonValueKind.Object) continue;
29              var values = new Dictionary<string, string>(StringComparer.OrdinalIgnoreCase);
30              foreach (var setting in section.Value.EnumerateObject())
31                  values[setting.Name] = setting.Value.ValueKind == JsonValueKind.String ? setting
        .Value.GetString()! : setting.Value.GetRawText();
32              sections[section.Name] = values;
33          }
34          return new ModuleSettings(sections);
35      }
```

Line 24 modules.json is edited by people, so comments and a stray trailing comma are allowed.

Line 25 Section and key names ignore case, so "to" and "To" are the same setting.

Line 31 Non-string values (numbers, booleans) are kept as their JSON text and parsed on request by `Get<T>`.

C# snack3-plugin-framework/src/SolutionFramework.Common/Configuration.cs, lines 40–48

```
40      private sealed class SectionConfig(Dictionary<string, string> values) : IModuleConfig
41      {
42          public string? Get(string key) => values.TryGetValue(key, out var value) ? value : null;
43
44          public T Get<T>(string key, T defaultValue) where T : IParsable<T> =>
45              values.TryGetValue(key, out var text) && T.TryParse(text, CultureInfo.InvariantCultu
        re, out var parsed)
46                  ? parsed
47                  : defaultValue;
48      }
```

Each module receives only its own section. It can't read, or come to depend on, another module's settings.

JSON snack3-plugin-framework/src/SolutionFramework.Host/modules.json, lines 1–12

```json
{
  // Each module reads only its own section, found by the module's Name.
  "DoorForcedEmail": {
    "To": "security-desk@example.com",
    // Don't send another email for the same door within this many seconds.
    "QuietSeconds": 300
  },
  "AfterHoursAutoLock": {
    "Start": "19:00",
    "End": "07:00"
  }
}
```

3.5 Loading a module

C# snack3-plugin-framework/src/SolutionFramework.Host/PluginLoadContext.cs, lines 1–31

```csharp
using System.Reflection;
using System.Runtime.Loader;

namespace SolutionFramework.Host;

/// <summary>
/// One load context per module. A module's own dependencies resolve from its folder (using its
/// .deps.json), so two modules can use different versions of the same library. Anything the
/// module's folder doesn't have, notably SolutionFramework.Abstractions, falls back to the host's
/// copy, which is what makes ISolutionModule the same type for the host and every module.
/// </summary>
public sealed class PluginLoadContext(string mainAssemblyPath)
    : AssemblyLoadContext(Path.GetFileNameWithoutExtension(mainAssemblyPath), isCollectible: true)
{
    private readonly AssemblyDependencyResolver _resolver = new(mainAssemblyPath);

    protected override Assembly? Load(AssemblyName assemblyName)
    {
        // Never load a second copy of the contract, even if a module shipped one by mistake.
        if (assemblyName.Name == typeof(ISolutionModule).Assembly.GetName().Name) return null;

        var path = _resolver.ResolveAssemblyToPath(assemblyName);
        return path is null ? null : LoadFromAssemblyPath(path);
    }

    protected override IntPtr LoadUnmanagedDll(string unmanagedDllName)
    {
        var path = _resolver.ResolveUnmanagedDllToPath(unmanagedDllName);
        return path is null ? IntPtr.Zero : LoadUnmanagedDllFromPath(path);
    }
}
```

Line 13 A collectible context can be unloaded once nothing references its types, which is what hot-reloading a module would need.

Line 15 `AssemblyDependencyResolver` reads the module's `.deps.json` to know which of its dependencies are in its folder.

Line 20 Returning `null` means "not mine": the runtime falls back to the default context, the host's copy. This line makes the guarantee explicit even if a module ships a copy by mistake.

C# snack3-plugin-framework/src/SolutionFramework.Host/ModuleLoader.cs, lines 17–65

```
17      public static LoadResult LoadFrom(string modulesFolder)
18      {
19          var modules = new List<LoadedModule>();
20          var problems = new List<string>();
21
22          if (!Directory.Exists(modulesFolder))
23              return new LoadResult(modules, [$"Modules folder not found: {modulesFolder}"]);
24
25          foreach (var folder in Directory.GetDirectories(modulesFolder).OrderBy(f => f, StringCom
    parer.Ordinal))
26          {
27              var name = Path.GetFileName(folder);
28              var dll = Path.Combine(folder, name + ".dll");
29              if (!File.Exists(dll))
30              {
31                  problems.Add($"{name}: expected {name}.dll in its folder");
32                  continue;
33              }
34
35              var context = new PluginLoadContext(dll);
36              Assembly assembly;
37              try
38              {
39                  assembly = context.LoadFromAssemblyPath(dll);
40              }
41              catch (Exception e) when (e is BadImageFormatException or FileLoadException)
42              {
43                  problems.Add($"{name}: not a .NET assembly ({e.Message})");
44                  context.Unload();
45                  continue;
46              }
47
48              var found = 0;
49              foreach (var type in ModuleTypes(assembly))
50              {
51                  try
52                  {
53                      modules.Add(new LoadedModule((ISolutionModule)Activator.CreateInstance(type)
    !, dll, context));
54                      found++;
55                  }
56                  catch (Exception e)
57                  {
58                      problems.Add($"{name}: couldn't create {type.FullName} ({(e.InnerException ?
    ? e).Message})");
59                  }
60              }
61              if (found == 0) problems.Add($"{name}: no public ISolutionModule with a parameterles
    s constructor");
62          }
63
64          return new LoadResult(modules, problems);
65      }
```

Line 28 Convention over configuration: the folder's name is the DLL's name, so there's no manifest to keep in sync.

Line 41 A file that isn't a .NET assembly becomes a reported problem. The context is unloaded so the failed attempt doesn't linger.

Line 53 `Activator` calls the parameterless constructor. The cast works only because of the shared contract.

Line 58 A constructor that throws arrives wrapped in `TargetInvocationException`; the inner exception is the useful message.

C# snack3-plugin-framework/src/SolutionFramework.Host/ModuleLoader.cs, lines 67–82

```
67      private static IEnumerable<Type> ModuleTypes(Assembly assembly)
68      {
69          Type?[] types;
70          try
71          {
72              types = assembly.GetExportedTypes();
73          }
74          catch (ReflectionTypeLoadException e)
75          {
76              types = e.Types;
77          }
78          return types.OfType<Type>().Where(t =>
79              t is { IsClass: true, IsAbstract: false } &&
80              typeof(ISolutionModule).IsAssignableFrom(t) &&
81              t.GetConstructor(Type.EmptyTypes) is not null);
82      }
```

`GetExportedTypes` lists public types. If some of an assembly's types can't load (a missing dependency), it throws `ReflectionTypeLoadException`, whose `Types` array still holds the ones that did, with nulls for the rest; `OfType<Type>()` drops the nulls.

3.6 Building modules into a folder

MSBuild snack3-plugin-framework/modules/DoorForcedEmail/DoorForcedEmail.csproj, lines 1–21

```
1  <Project Sdk="Microsoft.NET.Sdk">
2
3    <PropertyGroup>
4      <TargetFramework>net10.0</TargetFramework>
5      <ImplicitUsings>enable</ImplicitUsings>
6      <!-- A module's .deps.json lets the host's load context find the module's own dependencies.
         -->
7      <EnableDynamicLoading>true</EnableDynamicLoading>
8    </PropertyGroup>
9
10   <ItemGroup>
11     <!--
12       Compile against the contract, but don't copy it into the module's folder (Private=false):
13       at run time the module must use the host's copy, or ISolutionModule would be a different t
         ype.
14     -->
15     <ProjectReference Include="..\..\src\SolutionFramework.Abstractions\SolutionFramework.Abstra
         ctions.csproj">
16       <Private>false</Private>
17       <ExcludeAssets>runtime</ExcludeAssets>
18     </ProjectReference>
19   </ItemGroup>
20
21 </Project>
```

Line 7 Marks the project as a plugin: it gets a `.deps.json` and copies its dependencies to its output, everything the resolver needs.

Line 16 Don't copy the contract's DLL into the module's output...

Line 17 ...and don't list it as a runtime dependency. Compile against it, load the host's.

MSBuild snack3-plugin-framework/modules/Directory.Build.targets, lines 1–18

```
<Project>
  <!--
    Imported by every module project. After a module builds, its output (its DLL, its .deps.json and
    any libraries of its own) is copied to artifacts/modules/<ModuleName>/, which is where the host
    looks when run from source. Shipping a module means copying that one folder.
  -->
  <PropertyGroup>
    <ModulesFolder>$(MSBuildThisFileDirectory)..\artifacts\modules\$(AssemblyName)\</ModulesFolder>
  </PropertyGroup>

  <Target Name="CopyModuleToModulesFolder" AfterTargets="Build">
    <ItemGroup>
      <ModuleOutput Include="$(OutDir)**\*" />
    </ItemGroup>
    <Copy SourceFiles="@(ModuleOutput)" DestinationFiles="@(ModuleOutput->'$(ModulesFolder)%(RecursiveDir)%(Filename)%(Extension)')"
          SkipUnchangedFiles="true" />
  </Target>
</Project>
```

MSBuild imports `Directory.Build.targets` into every project below its folder, so every module gets this step without repeating it. After each build, the module's output lands in `artifacts/modules/<Name>/`, which is the folder the host searches when run from source.

3.7 Routing events

C# snack3-plugin-framework/src/SolutionFramework.Host/EventRouter.cs, lines 35–55

```
    /// <summary>Initializes the module and adds it. A module whose Initialize throws is not added.</summary>
    public bool Add(ISolutionModule module)
    {
        var context = new ModuleContext(
            new SourceLogger(module.Name, _log, _time),
            _settings.For(module.Name),
            new CommandSink(module.Name, _pending),
            _time);
        try
        {
            module.Initialize(context);
        }
        catch (Exception e)
        {
            _hostLog.Error($"{module.Name} failed to initialize and won't receive events", e);
            return false;
        }
        _slots.Add(new Slot(module));
        _hostLog.Info($"Loaded {module.Name}");
        return true;
    }
```

Each module gets its own context: a logger with its name, its own settings section, and a command sink that remembers who sent what. A module whose `Initialize` throws (a missing setting, say) is logged and left out, and the rest load normally.

C# snack3-plugin-framework/src/SolutionFramework.Host/EventRouter.cs, lines 57–85

```
57      public void Publish(SecurityEvent securityEvent)
58      {
59          foreach (var slot in _slots.Where(s => s.Enabled))
60          {
61              try
62              {
63                  slot.Module.OnEvent(securityEvent);
64                  slot.ConsecutiveFailures = 0;
65              }
66              catch (Exception e)
67              {
68                  slot.ConsecutiveFailures++;
69                  _hostLog.Error($"{slot.Module.Name} failed on {securityEvent.Kind} at {securityE
          vent.DoorId}", e);
70                  if (slot.ConsecutiveFailures >= MaxConsecutiveFailures)
71                  {
72                      slot.Enabled = false;
73                      _hostLog.Warn($"{slot.Module.Name} failed {MaxConsecutiveFailures} times in
          a row and has been switched off");
74                  }
75              }
76          }
77
78          // Carry out commands now, after every module has seen the event. A command can produce
          a
79          // new event (a door locking); that's published too, so modules see the effect of each o
          ther.
80          while (_pending.TryDequeue(out var command))
81          {
82              var result = _doors.Execute(command);
83              if (result is not null) Publish(result);
84          }
85      }
```

Line 59 Every running module, in the order they were added. Sequential delivery keeps reasoning simple: a module never sees two events at once.

Line 64 One success resets the count, so a module that fails occasionally keeps running.

Line 70 A **circuit breaker**: three failures in a row and the module is switched off, so a broken module can't flood the log with the same exception forever.

Line 80 Commands run only after every module has seen the event, so no module sees the world change halfway through a delivery.

Line 83 The door's answer (`DoorLocked`) is published like any event, so modules see each other's effects without knowing about each other.

C# snack3-plugin-framework/src/SolutionFramework.Host/EventRouter.cs, lines 96–103

```
96      private sealed class CommandSink(string module, Queue<DeviceCommand> pending) : ICommandSink
97      {
98          public void Send(DeviceCommand command)
99          {
100             ArgumentNullException.ThrowIfNull(command);
101             pending.Enqueue(command with { Reason = $"{command.Reason} (requested by {module})"
          });
102         }
103     }
```

`command with { Reason = ... }` copies the record with one property changed: the host appends which module asked, which shows up in the door's log line.

3.8 The two modules

C# snack3-plugin-framework/modules/DoorForcedEmail/DoorForcedEmailModule.cs, lines 22–28

```
22      public void Initialize(IModuleContext context)
23      {
24          _context = context;
25          _to = context.Config.Get("To") ?? throw new InvalidOperationException("DoorForcedEmail n
          eeds a \"To\" address in modules.json.");
26          _quiet = TimeSpan.FromSeconds(context.Config.Get("QuietSeconds", 300));
27          context.Logger.Info($"Will email {_to} when a door is forced (at most once per door ever
          y {_quiet.TotalMinutes:0} min)");
28      }
```

A module validates its own configuration and fails `Initialize` with a message that says what to fix. The router turns that into a log line and leaves the module out.

C# snack3-plugin-framework/modules/DoorForcedEmail/DoorForcedEmailModule.cs, lines 30–43

```
30      public void OnEvent(SecurityEvent e)
31      {
32          if (e.Kind != SecurityEventKind.DoorForced) return;
33
34          if (_lastSent.TryGetValue(e.DoorId, out var last) && e.OccurredAt - last < _quiet)
35          {
36              _context.Logger.Info($"{e.DoorName} forced again; already emailed at {last:HH:mm}");
37              return;
38          }
39
40          _lastSent[e.DoorId] = e.OccurredAt;
41          Sent++;
42          _context.Logger.Warn($"EMAIL to {_to}: \"Door forced: {e.DoorName} ({e.Location}) at {e.
          OccurredAt:HH:mm}\"");
43      }
```

A forced door often reports several times in a row. The module remembers when it last emailed about each door and stays quiet for `QuietSeconds`: the alert people actually want, not one per sensor bounce.

C# snack3-plugin-framework/modules/AfterHoursAutoLock/AfterHoursAutoLockModule.cs, lines 27–39

```
27      /// <summary>Is this time inside the after-hours window? Handles windows that cross midnight
          .</summary>
28      public bool IsAfterHours(TimeOnly time) =>
29          Start <= End
30              ? time >= Start && time < End          // e.g. 01:00 to 05:00
31              : time >= Start || time < End;         // e.g. 19:00 to 07:00
32
33      public void OnEvent(SecurityEvent e)
34      {
35          if (e.Kind is not (SecurityEventKind.DoorUnlocked or SecurityEventKind.DoorHeldOpen)) re
          turn;
36          if (!IsAfterHours(TimeOnly.FromDateTime(e.OccurredAt.DateTime))) return;
37
38          _context.Commands.Send(new LockDoorCommand(e.DoorId, $"{e.Kind} after hours at {e.Occurr
          edAt:HH:mm}"));
39      }
```

The classic bug in after-hours rules is the window that crosses midnight. 19:00 to 07:00 isn't "after 19:00 *and* before 07:00" (no time is both) but "after 19:00 *or* before 07:00". The method handles both shapes, and tests check 18:59, 19:00, midnight, 06:59 and 07:00 for one and a

same-day window for the other.

C# snack3-plugin-framework/modules/AfterHoursAutoLock/AfterHoursAutoLockModule.cs, lines 33–39

```
33      public void OnEvent(SecurityEvent e)
34      {
35          if (e.Kind is not (SecurityEventKind.DoorUnlocked or SecurityEventKind.DoorHeldOpen)) re
          turn;
36          if (!IsAfterHours(TimeOnly.FromDateTime(e.OccurredAt.DateTime))) return;
37
38          _context.Commands.Send(new LockDoorCommand(e.DoorId, $"{e.Kind} after hours at {e.Occurr
          edAt:HH:mm}"));
39      }
```

`e.Kind is not (A or B)` is a combined pattern: the module only cares about unlocked and held-open doors.

3.9 The host program

C# snack3-plugin-framework/src/SolutionFramework.Host/SimulatedBuilding.cs, lines 66–75

```
66  /// <summary>A clock the simulation moves by hand. Time zone is the computer's.</summary>
67  public sealed class ManualTimeProvider(DateTimeOffset start) : TimeProvider
68  {
69      private DateTimeOffset _now = start;
70
71      public override DateTimeOffset GetUtcNow() => _now.ToUniversalTime();
72      public override TimeZoneInfo LocalTimeZone => TimeZoneInfo.Utc;
73
74      public void Advance(TimeSpan by) => _now += by;
75  }
```

The simulation's clock: it only moves when told. `SimulatedBuilding.Next` advances it two to eleven minutes per event, so an evening from 17:30 passes in a second and the 19:00 rule kicks in partway.

C# snack3-plugin-framework/src/SolutionFramework.Host/Program.cs, lines 4–29

```
4   // solution-host [--modules <folder>] [--config <modules.json>] [--events <count>] [--seed <n>]
5   //
6   // Loads every module, then plays a simulated evening at the building (from 17:30, a few minutes
        per
7   // event) through them. Each module's log lines are prefixed with its name.
8   var options = Options.Parse(args);
9   var log = new ConsoleLogSink();
10  var time = new ManualTimeProvider(new DateTimeOffset(2026, 11, 12, 17, 30, 0, TimeSpan.Zero));
11  var building = new SimulatedBuilding(new Random(options.Seed), time, log);
12  var router = new EventRouter(log, time, ModuleSettings.Load(options.Config), building);
13
14  Console.WriteLine($"Modules folder: {options.Modules}");
15  var loaded = ModuleLoader.LoadFrom(options.Modules);
16  foreach (var problem in loaded.Problems) Console.WriteLine($"  problem: {problem}");
17  foreach (var module in loaded.Modules) router.Add(module.Module);
18  if (router.RunningModules.Count == 0)
19  {
20      Console.WriteLine("No modules loaded. Build the solution first (dotnet build), which copies
          them into the modules folder.");
21      return 1;
22  }
23
24  for (var i = 0; i < options.Events; i++) router.Publish(building.Next());
25
26  Console.WriteLine();
```

```
Console.WriteLine($"Done: {options.Events} events. Running: {string.Join(", ", router.RunningMod
      ules)}." +
                  (router.DisabledModules.Count > 0 ? $" Switched off: {string.Join(", ", router
      .DisabledModules)}." : ""));
return 0;
```

Top-level statements: the whole program, no `Main` method. Problems loading modules are printed, not thrown. With no modules at all, the exit code is 1 and the message says how to get some.

3.10 Testing a framework

C# snack3-plugin-framework/tests/SolutionFramework.Tests/ModuleLoaderTests.cs, lines 39–50

```
    public void Modules_share_the_hosts_copy_of_the_contract()
    {
        // If a module loaded its own SolutionFramework.Abstractions, its class would implement
      a
        // different ISolutionModule and the cast in the loader would fail.
        var result = ModuleLoader.LoadFrom(ModulesFolder);
        var contract = typeof(ISolutionModule).Assembly;

        Assert.All(result.Modules, m => Assert.Contains(typeof(ISolutionModule), m.Module.GetTyp
      e().GetInterfaces()));
        Assert.Same(AssemblyLoadContext.Default, AssemblyLoadContext.GetLoadContext(contract));
        Assert.All(Directory.GetDirectories(ModulesFolder), folder =>
            Assert.False(File.Exists(Path.Combine(folder, contract.GetName().Name + ".dll")), $"
      {folder} ships its own copy of the contract"));
    }
```

The type-identity guarantee from 3.2 as a test: every loaded module implements the host's `ISolutionModule`, the contract lives in the default context, and no module folder ships a copy.

C# snack3-plugin-framework/tests/SolutionFramework.Tests/ModuleLoaderTests.cs, lines 53–73

```
    public void Problems_are_reported_without_stopping_the_load()
    {
        var folder = Directory.CreateTempSubdirectory("modules-").FullName;
        try
        {
            Directory.CreateDirectory(Path.Combine(folder, "Empty"));
            Directory.CreateDirectory(Path.Combine(folder, "Junk"));
            File.WriteAllText(Path.Combine(folder, "Junk", "Junk.dll"), "not really a DLL");
            CopyModule("DoorForcedEmail", folder);

            var result = ModuleLoader.LoadFrom(folder);

            Assert.Equal("DoorForcedEmail", Assert.Single(result.Modules).Module.Name);
            Assert.Contains(result.Problems, p => p.StartsWith("Empty: expected Empty.dll"));
            Assert.Contains(result.Problems, p => p.StartsWith("Junk: not a .NET assembly"));
        }
        finally
        {
            try { Directory.Delete(folder, recursive: true); } catch (IOException) { } catch (Un
      authorizedAccessException) { }
        }
    }
```

C# snack3-plugin-framework/tests/SolutionFramework.Tests/EventRouterTests.cs, lines 47–58

```
47      public void A_module_that_fails_three_times_in_a_row_is_switched_off()
48      {
49          var broken = new ScriptedModule("Broken") { ThrowWhen = _ => true };
50          _router.Add(broken);
51          _router.Add(new ScriptedModule("Healthy"));
52
53          for (var i = 0; i < 5; i++) _router.Publish(Events.At(SecurityEventKind.AccessGranted, "
          09:00"));
54
55          Assert.Equal(EventRouter.MaxConsecutiveFailures, broken.Seen.Count);
56          Assert.Equal(new[] { "Healthy" }, _router.RunningModules);
57          Assert.Equal(new[] { "Broken" }, _router.DisabledModules);
58      }
```

C# snack3-plugin-framework/tests/SolutionFramework.Tests/ModuleLoaderTests.cs, lines 84–99

```
84      public void An_evening_at_the_building_end_to_end()
85      {
86          // The whole host: real modules from their folders, the shipped modules.json, the simula
          tor.
87          var log = new MemoryLogSink();
88          var time = new ManualTimeProvider(new DateTimeOffset(2026, 11, 12, 17, 30, 0, TimeSpan.Z
          ero));
89          var building = new SimulatedBuilding(new Random(2026), time, log);
90          var router = new EventRouter(log, time, ModuleSettings.Load(Path.Combine(AppContext.Base
          Directory, "modules.json")), building);
91          foreach (var loaded in ModuleLoader.LoadFrom(ModulesFolder).Modules) Assert.True(router.
          Add(loaded.Module));
92
93          for (var i = 0; i < 200; i++) router.Publish(building.Next());
94
95          Assert.Equal(2, router.RunningModules.Count);
96          Assert.Contains(log.Entries, e => e.Source == "DoorForcedEmail" && e.Message.StartsWith(
          "EMAIL to security-desk@example.com"));
97          Assert.Contains(log.Entries, e => e.Source == "doors" && e.Message.Contains("locked") &&
           e.Message.Contains("requested by AfterHoursAutoLock"));
98          Assert.DoesNotContain(log.Entries, e => e.Level == LogLevel.Error);
99      }
```

The end-to-end test runs the whole host as `Program.cs` does (real module folders, the shipped `modules.json`, the simulator) for 200 events, and checks an email went out, a door was locked at a module's request, and nothing logged an error.

3.11 Interview questions

Why does the contract live in its own assembly?

So it can be shared by exactly one copy. If the host and a module each loaded their own, ISolutionModule would be two different types and casting a module to it would fail. A small, separate contract assembly that modules compile against but don't ship makes that impossible.

What does an AssemblyLoadContext give you that Assembly.LoadFrom doesn't?

Isolation and control: each module's dependencies resolve from its own folder (so modules can use different versions of a library), you decide which assemblies are shared, and a collectible context can be unloaded.

A module throws on every event. What happens?

Each exception is logged and the other modules still get the event. After three failures in a row the router switches the module off. A single success resets the count.

Why commands instead of letting the module lock the door itself?

The host stays in control of hardware: it can validate, log, rate-limit or refuse. Modules stay testable (a test checks the command, no door needed), and the result is published as an event other modules can see.

Is adding a method to ISolutionModule a breaking change?

Yes. Every existing module fails to load (it doesn't implement the new member) or to compile. Under Semantic Versioning that's a new major version of the contract. Default interface methods, or a second optional interface, are the usual ways to extend without breaking.

How would you hot-reload a module?

Watch the folder; when it changes, remove the module from the router, drop every reference to its types, unload its collectible context, and load the new build into a new context.

3.12 Exercises

A third module

Write a module that counts denied swipes per door and warns after five in ten minutes.

Hint. Copy DoorForcedEmail's project; keep a queue of timestamps per door. Test it with `FakeContext`.

Module versions

Have each module declare which contract version it was built for, and refuse to load modules built for a newer one.

Hint. An assembly attribute in the contract, read by the loader with `GetCustomAttribute`; compare major versions.

Hot reload

Reload a module when its folder changes, without restarting the host.

Hint. `FileSystemWatcher`, `AssemblyLoadContext.Unload`, and a test that the old context is collected (a `WeakReference` to it goes dead after `GC.Collect`).

Async modules

Real email sending is async. Add `OnEventAsync` without breaking existing modules.

Hint. A default interface method that calls `OnEvent`, or a separate `IAsyncSolutionModule` the router checks for. Which one is a minor version, and which a major?

Part 4

Web Dashboard

Snack 2's events on a web page: an ASP.NET Core minimal API that reuses Snack 2's repository unchanged, and a React page with a status filter. Small on purpose; the interesting parts are reuse, validation at the edge, and tests that control the world the API sees.

4.1 What it does

The API serves the events, filtered by outcome and search text, and both of Snack 2's reports, as JSON. The page shows them, refreshes every three seconds, and filters with a row of buttons (all, granted, denied, status changes) and a search box. Pointed at Snack 2's database, it shows exactly what the WPF app shows; on its own, it seeds an in-memory store with a demo day and keeps adding events.

4.2 Minimal APIs and dependency injection

A **minimal API** maps a route straight to a function: `api.MapGet("/events", handler)`. ASP.NET Core inspects the handler's parameters and fills them: from the query string for simple types (`string? outcome`, `int? limit`), and from the **dependency injection** container for registered services (`IAccessEventRepository repo`). No controllers, no attributes.

The container is set up before `builder.Build()`: `AddSingleton<IClock, SystemClock>()` says "when something asks for `IClock`, give it the one `SystemClock`". Tests use the same mechanism to swap in a fixed clock and a store they filled themselves.

4.3 Reuse: one project reference

MSBuild snack4-web-dashboard/api/Dashboard.Api/Dashboard.Api.csproj, lines 1–13

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <!-- A small web API over Snack 2's repository: same events, same reports, as JSON. -->
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <ProjectReference Include="..\..\..\snack2-access-event-log\src\AccessLog.Core\AccessLog.Cor
      e.csproj" />
  </ItemGroup>

</Project>
```

The whole of Snack 2's data layer comes in through one project reference. `AccessLog.Core` targets `net48` and `net10.0`; this project is `net10.0`, so it gets that build. The SQL, the reports and the in-memory store are the same code the WPF app runs.

4.4 Program.cs

C# snack4-web-dashboard/api/Dashboard.Api/Program.cs, lines 9–21

```
 9  // Where events come from: Snack 2's SQL Server database if a connection string is set
10  // (ConnectionStrings__AccessLog), otherwise an in-memory store filled with a demo day.
11  var connection = builder.Configuration.GetConnectionString("AccessLog");
12  builder.Services.AddSingleton<IClock, SystemClock>();
13  if (string.IsNullOrWhiteSpace(connection))
14  {
15      builder.Services.AddSingleton<IAccessEventRepository, InMemoryAccessEventRepository>();
16      builder.Services.AddSingleton<StoreInfo>(new StoreInfo("In memory (demo data)"));
17  }
18  else
19  {
20      builder.Services.AddSingleton<IAccessEventRepository>(_ => new SqlServerAccessEventRepositor
        y(connection));
21      builder.Services.AddSingleton<StoreInfo>(new StoreInfo("SQL Server"));
```

Line 11 Reads `ConnectionStrings:AccessLog` from appsettings.json, or the environment variable `ConnectionStrings__AccessLog` (the double underscore stands for the colon), which wins.

Line 20 A factory registration, because the constructor needs a string the container can't provide.

C# snack4-web-dashboard/api/Dashboard.Api/Program.cs, lines 24–34

```
24  // Demo:LiveEvents (default true) keeps adding events, so the page has something to update.
25  builder.Services.AddHostedService<DemoActivity>();
26  builder.Services.ConfigureHttpJsonOptions(o => o.SerializerOptions.Converters.Add(new JsonString
        EnumConverter()));
27
28  var app = builder.Build();
29
30  if (!string.IsNullOrWhiteSpace(connection))
31      await SqlServerAccessEventRepository.EnsureDatabaseAsync(connection);
32
33  app.UseDefaultFiles();
34  app.UseStaticFiles(); // the built React app (npm run build in web/ writes to wwwroot)
```

Line 26 Enums go out as text: `"BadgeSwipe"` rather than `0`. The JSON reads well, and the TypeScript types can be string unions.

Line 31 Same schema script as the WPF app, so either can create the database first.

Line 33 Serves `index.html` for `/`; with `UseStaticFiles`, the built React app in `wwwroot` is served from the same origin as the API.

C# snack4-web-dashboard/api/Dashboard.Api/Program.cs, lines 40–43

```
40  api.MapGet("/events", async (IAccessEventRepository repo, string? outcome, string? search, int?
        limit) =>
41  {
42      if (!Filters.TryParseOutcome(outcome, out var filter))
43          return Results.BadRequest(new { error = "outcome must be all, granted, denied or status"
            });
```

Line 42 **Validation at the edge**: bad input is rejected here, with a message, before it reaches the repository.

Line 44 A relational pattern on `int?`: null (not given) matches neither branch, so the default applies; 0 or 501 is refused. Capping the limit stops one request asking for a million rows.

Line 48 Snack 2's `AccessEventView` objects, serialized as camelCase JSON, including the computed `Outcome`.

C# snack4-web-dashboard/api/Dashboard.Api/Program.cs, lines 51–59

```
51  api.MapGet("/reports/denied-by-door", async (IAccessEventRepository repo, IClock clock, int? hou
        rs) =>
52      hours is < 1 or > 168
53          ? Results.BadRequest(new { error = "hours must be between 1 and 168" })
54          : Results.Ok(await repo.DeniedByDoorAsync(clock.UtcNow.AddHours(-(hours ?? 24)))));
55
56  api.MapGet("/reports/noisiest", async (IAccessEventRepository repo, IClock clock, int? hours, in
        t? top) =>
57      hours is < 1 or > 168 || top is < 1 or > 50
58          ? Results.BadRequest(new { error = "hours must be 1-168 and top 1-50" })
59          : Results.Ok(await repo.NoisiestDevicesAsync(clock.UtcNow.AddHours(-(hours ?? 24)), top
          ?? 5)));
```

The same shape for both reports. Errors always use one shape, `{ "error": "..." }`, so the page can show the message as it is.

C# snack4-web-dashboard/api/Dashboard.Api/Program.cs, lines 64–67

```
64  public sealed record StoreInfo(string Description);
65
66  // Lets the tests start this app with WebApplicationFactory<Program>.
67  public partial class Program { }
```

Top-level statements compile into a class called `Program` that's internal; declaring it `public partial` lets the test project name it in `WebApplicationFactory<Program>`.

4.5 Filters and the demo

C# snack4-web-dashboard/api/Dashboard.Api/Filters.cs, lines 16–24

```
16      public static bool TryParseOutcome(string? text, out Outcome outcome)
17      {
18          if (string.IsNullOrWhiteSpace(text))
19          {
20              outcome = Outcome.All;
21              return true;
22          }
23          return Enum.TryParse(text, ignoreCase: true, out outcome) && Enum.IsDefined(outcome);
24      }
```

`Enum.TryParse` also accepts numbers: "7" parses into a value with no name. `Enum.IsDefined` rejects it, so only the four names are valid. A test checks exactly this case.

C# snack4-web-dashboard/api/Dashboard.Api/DemoActivity.cs, lines 13–27

```
13      protected override async Task ExecuteAsync(CancellationToken stoppingToken)
14      {
15          if (!config.GetValue("Demo:Seed", true)) return;
16
17          var simulator = new AccessSimulator(repository, new Random(), clock);
18          await simulator.SeedAsync();
```

```
19          if ((await repository.SearchAsync(new AccessLog.Core.Model.EventQuery { Limit = 1 })).Co
        unt == 0)
20              await simulator.BackfillAsync(hours: 24, events: 400);
21          log.LogInformation("Demo building ready");
22
23          if (!config.GetValue("Demo:LiveEvents", true)) return;
24          using var timer = new PeriodicTimer(TimeSpan.FromSeconds(2));
25          while (await timer.WaitForNextTickAsync(stoppingToken))
26              await simulator.StepAsync();
27      }
```

Line 15 Switchable from configuration; the tests turn it off so they control every row.

Line 24 `PeriodicTimer` is the async timer: each tick is awaited, so a slow step can't overlap the next, and the `stoppingToken` ends the loop cleanly when the app shuts down.

A `BackgroundService` runs alongside the web server for the app's lifetime. Here it reuses Snack 2's simulator, so the web demo and the WPF demo produce the same kind of events.

4.6 The page

TypeScript snack4-web-dashboard/web/src/types.ts, lines 1–32

```
1  /** The shapes the API returns (Snack 2's model, serialized in camelCase with enums as text). */
2  export type EventKind = "BadgeSwipe" | "StatusChange";
3
4  export interface AccessEvent {
5    id: number;
6    occurredAtUtc: string;
7    deviceId: number;
8    deviceName: string;
9    location: string;
10   cardholderId: number | null;
11   cardholderName: string | null;
12   kind: EventKind;
13   granted: boolean | null;
14   detail: string | null;
15   outcome: string;
16 }
17
18 export interface DeniedByDoorRow {
19   deviceId: number;
20   deviceName: string;
21   location: string;
22   denied: number;
23 }
24
25 export interface NoisyDeviceRow {
26   deviceId: number;
27   deviceName: string;
28   kind: "Door" | "Camera";
29   events: number;
30 }
31
32 export type Outcome = "all" | "granted" | "denied" | "status";
```

TypeScript types mirroring the JSON. Because enums arrive as text, `kind` is a union of two strings: a typo in a comparison is a compile error.

TypeScript snack4-web-dashboard/web/src/filters.ts, lines 10–19

```
10  /** The query string for /api/events. Defaults are left out, so the URL stays short. */
11  export function eventsQuery(outcome: Outcome, search: string, limit = 100): string {
12    const params = new URLSearchParams();
13    if (outcome !== "all") params.set("outcome", outcome);
14    const text = search.trim();
15    if (text) params.set("search", text);
16    if (limit !== 100) params.set("limit", String(limit));
17    const query = params.toString();
18    return query ? `?${query}` : "";
19  }
```

`URLSearchParams` builds the query string and encodes it (spaces become `+`, `&` becomes `%26`), so a search for "a&b;" can't add a parameter. Defaults are left out to keep URLs short.

TypeScript snack4-web-dashboard/web/src/filters.ts, lines 28–36

```
28  /** "just now", "5 min ago", "3 h ago", or the date for anything older than a day. */
29  export function relativeTime(isoUtc: string, now: Date): string {
30    const then = new Date(isoUtc.endsWith("Z") ? isoUtc : `${isoUtc}Z`);
31    const seconds = Math.round((now.getTime() - then.getTime()) / 1000);
32    if (seconds < 60) return "just now";
33    if (seconds < 3600) return `${Math.floor(seconds / 60)} min ago`;
34    if (seconds < 86400) return `${Math.floor(seconds / 3600)} h ago`;
35    return then.toISOString().slice(0, 10);
36  }
```

The API sends UTC; this treats a time without a zone as UTC too, so the page and the database agree no matter where the browser is.

TSX snack4-web-dashboard/web/src/App.tsx, lines 18–36

```
18    const load = useCallback(async () => {
19      try {
20        const [e, d, n] = await Promise.all([api.events(outcome, search), api.deniedByDoor(), api.
          noisiest()]);
21        setEvents(e);
22        setDenied(d);
23        setNoisy(n);
24        setNow(new Date());
25        setError(null);
26      } catch (err) {
27        setError(err instanceof Error ? err.message : String(err));
28      }
29    }, [outcome, search]);
30
31    // Reload when the filters change, and every few seconds to pick up new events.
32    useEffect(() => {
33      load();
34      const timer = setInterval(load, REFRESH_MS);
35      return () => clearInterval(timer);
36    }, [load]);
```

Line 20 Three requests in parallel; the page updates once all three are back.

Line 29 `useCallback` makes a new `load` only when the filters change...

Line 34 ...and this effect depends on `load`, so changing a filter reloads at once and restarts the timer. The cleanup function clears the old timer, so there's never more than one.

TSX snack4-web-dashboard/web/src/App.tsx, lines 50–57

```
50          <div className="segmented" role="radiogroup" aria-label="Outcome">
51            {OUTCOMES.map((o) => (
52              <button key={o.value} role="radio" aria-checked={outcome === o.value}
53                      className={outcome === o.value ? "on" : ""} onClick={() => setOutcome(o.valu
        e)}>
54                {o.label}
55              </button>
56            ))}
57          </div>
```

A row of buttons acting as one choice. `role="radiogroup"` and `aria-checked` tell screen readers it's a single-choice control, which a plain row of buttons wouldn't.

TypeScript snack4-web-dashboard/web/vite.config.ts, lines 1–15

```
1  import { defineConfig } from "vite";
2  import react from "@vitejs/plugin-react";
3
4  // npm run dev: the page on :5173, forwarding /api to the API on :5080.
5  // npm run build: writes the page into the API's wwwroot, so the API serves both on one origin.
6  export default defineConfig({
7    plugins: [react()],
8    server: {
9      proxy: { "/api": "http://localhost:5080" },
10   },
11   build: {
12     outDir: "../api/Dashboard.Api/wwwroot",
13     emptyOutDir: true,
14   },
15 });
```

In development, Vite's proxy forwards `/api` to the API, so the page and the API look like one origin and there's no CORS to set up. `npm run build` writes the page into the API's `wwwroot`, so in production they really are one origin.

4.7 Tests that control the world

C# snack4-web-dashboard/tests/Dashboard.Api.Tests/DashboardApiTests.cs, lines 46–56

```
46          _factory = new WebApplicationFactory<Program>().WithWebHostBuilder(b =>
47          {
48              b.UseSetting("Demo:Seed", "false");
49              b.UseSetting("ConnectionStrings:AccessLog", "");
50              b.ConfigureServices(services =>
51              {
52                  services.AddSingleton<IAccessEventRepository>(_repo);
53                  services.AddSingleton<IClock>(new FixedClock());
54              });
55          });
56          _client = _factory.CreateClient();
```

Line 48 No random demo data: the test's four events are the only ones.

Line 52 Registered after the app's own registrations, so it wins: the API reads the store the test filled.

Line 53 "The last 24 hours" is measured from a fixed instant, so a 30-hour-old event is always outside it.

C# snack4-web-dashboard/tests/Dashboard.Api.Tests/DashboardApiTests.cs, lines 83–93

```
83      public async Task Events_are_newest_first_in_camel_case_with_enums_as_text()
84      {
85          var events = await GetJson("/api/events");
86
87          Assert.Equal(4, events.GetArrayLength());
88          var newest = events[0];
89          Assert.Equal("Roof access", newest.GetProperty("deviceName").GetString());
90          Assert.Equal("StatusChange", newest.GetProperty("kind").GetString());
91          Assert.Equal("Online → Alarm", newest.GetProperty("outcome").GetString());
92          Assert.Equal(JsonValueKind.Null, newest.GetProperty("granted").ValueKind);
93      }
```

This tests the JSON *contract*, not just the data: property names in camelCase, enums as text, null as null. Those are what the TypeScript types rely on, so a serializer setting changed by accident would fail here rather than in the browser.

C# snack4-web-dashboard/tests/Dashboard.Api.Tests/DashboardApiTests.cs, lines 113–122

```
113     [InlineData("/api/events?outcome=maybe")]
114     [InlineData("/api/events?limit=0")]
115     [InlineData("/api/events?limit=501")]
116     [InlineData("/api/reports/denied-by-door?hours=0")]
117     [InlineData("/api/reports/noisiest?top=100")]
118     public async Task Bad_parameters_get_400_with_a_message(string url)
119     {
120         var response = await _client.GetAsync(url, Ct);
121         Assert.Equal(HttpStatusCode.BadRequest, response.StatusCode);
122         Assert.True((await response.Content.ReadFromJsonAsync<JsonElement>(Ct)).TryGetProperty("
          error", out _));
```

TypeScript snack4-web-dashboard/web/tests/filters.test.ts, lines 16–18

```
16  test("eventsQuery encodes characters that mean something in a URL", () => {
17    assert.equal(eventsQuery("all", "a&b=c"), "?search=a%26b%3Dc");
18  });
```

The page's logic is tested in Node with `node:test`, the same way as the robot arm, with no browser. `tests/api.test.ts` replaces `fetch` to check the URLs the client calls and that an error response throws the API's own message.

4.8 Interview questions

How does a minimal API handler get its arguments?

By parameter: simple types bind from the route and query string, complex types from the JSON body, and registered services from dependency injection. Optional query values are nullable parameters.

How do you test an ASP.NET Core API end to end without a server?

WebApplicationFactory runs the real app in memory with an in-memory HTTP client. You can override settings and services (here: no demo data, a filled store, a fixed clock) and assert on real HTTP responses.

Why send enums as strings?

Readable JSON, stable when enum members are reordered, and TypeScript can model them as string unions. Numbers silently change meaning if someone inserts a member in the middle of the enum.

Why serve the React build from the API?

One origin: no CORS configuration, one deployment, relative URLs work everywhere. In development the Vite proxy gives the same effect.

What would make this production-ready?

Authentication (operators only), paging instead of a fixed limit, server-sent events or SignalR instead of polling every three seconds, and output caching for the reports.

4.9 Exercises

Live updates

Replace polling with server-sent events.

Hint. An endpoint returning `text/event-stream`; the simulator or the repository raises an event when something is recorded. `EventSource` in the page.

Date range

Add from/to pickers to the page.

Hint. The API passes them to `EventQuery.FromUtc/ToUtc`; convert the browser's local dates to UTC before sending, and validate from < to at the edge.

A device page

Click a device to see its history and both reports for it alone.

Hint. Add `/api/devices/{id}/events`; `EventQuery.DeviceId` already exists, and the device index makes it fast.

Part 5

Keeping it working

The same three ideas as the robot arm's Appendix C, applied to a .NET solution: dependencies pinned by lock files, versions that mean something, and CI that builds and tests everything on every push, on Windows for the WPF apps and on Linux for SQL Server. Plus the real problems those checks caught while the snacks were built.

5.1 One build for four snacks

MSBuild Directory.Build.props, lines 1–12

```
<Project>
  <!-- Settings shared by every project under genetec/. -->
  <PropertyGroup>
    <Nullable>enable</Nullable>
    <LangVersion>latest</LangVersion>
    <!-- Write packages.lock.json next to each project: the exact version of every package. -->
    <RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>
    <!-- The version of the snacks, following Semantic Versioning (see CHANGELOG.md). -->
    <Version>1.0.0</Version>
    <Authors>Kyle Gartner</Authors>
  </PropertyGroup>
</Project>
```

MSBuild imports `Directory.Build.props` into every project below it, before the project's own settings. One place sets nullable reference types, the language version, lock files and the version number for all fourteen projects; a project can still override any of them.

JSON global.json, lines 1–9

```
{
  "sdk": {
    "version": "10.0.100",
    "rollForward": "latestFeature"
  },
  "test": {
    "runner": "Microsoft.Testing.Platform"
  }
}
```

Line 3 The minimum SDK...

Line 4 ...and any later feature band of .NET 10 (10.0.2xx, 10.0.3xx) is accepted, but not .NET 11. Everyone, including CI, builds with a .NET 10 SDK.

Line 7 xUnit v3's current releases run on Microsoft.Testing.Platform. On the .NET 10 SDK, `dotnet test` has to be told to use it, or it fails with "Testing with VSTest target is no longer supported". That's exactly how the first CI run failed.

5.2 Lock files

Every project has a `packages.lock.json` beside it, written because `RestorePackagesWithLockFile` is on. It records the exact version and content hash of every package the project uses, direct and transitive, per target framework (Snack 2's core has two sections, `net48` and `net10.0`). It's NuGet's `package-lock.json`.

`dotnet restore --locked-mode` installs exactly what the lock file records and fails if the project asks for something else, like `npm ci`. But it has a gap: a project with *no* lock file restores happily. CI closes it with a check of its own:

YAML ../.github/workflows/genetec.yml, lines 29–38

```yaml
      # --locked-mode only enforces a lock file that exists, so check every project has one.
      - name: Every project has a committed lock file
        shell: bash
        run: |
          missing=0
          for proj in $(git ls-files '*.csproj'); do
            lock="$(dirname "$proj")/packages.lock.json"
            git ls-files --error-unmatch "$lock" > /dev/null 2>&1 || { echo "::error file=$proj:
:no committed $lock"; missing=1; }
          done
          exit $missing
```

This was added after the first Genetec run went green with no lock files at all: `--locked-mode` had nothing to enforce. A check that can't fail isn't a check.

Exact versions in the project files

The lock files were generated by restoring with floating versions (`Version="*"`, "the newest"), and then every project file was pinned to the exact version the lock resolved, with the lock files updated to match. A project file should say what's actually used; a `*` in it means the next restore without a lock file could pull something new.

5.3 A vulnerable transitive dependency

The first restore of Snack 2 printed warnings `NU1902` and `NU1903`: `System.Security.Cryptography.Pkcs` 8.0.1 and 9.0.18 have known moderate and high severity vulnerabilities. Snack 2 never mentions Pkcs. It comes in through `Microsoft.Data.SqlClient`, which depends on it: a **transitive dependency**.

NuGet resolves each package to the lowest version that satisfies every request (unlike npm, which takes the newest allowed). So adding a direct reference to a patched version raises the floor, and NuGet picks it for the whole graph. That's the override in `AccessLog.Core.csproj`. The same thing happened in Full Stack's dotnet-sqlserver, where EF Core 10.0.0 pulled in the vulnerable Pkcs; there, the EF Core 10.0.15 patch release depended on a fixed version, so a patch bump was enough.

YAML ../.github/workflows/genetec.yml, lines 43–47

```yaml
      - name: Vulnerable packages
        shell: bash
        run: |
          dotnet list Genetec.slnx package --vulnerable --include-transitive | tee vulnerable.tx
t
          ! grep -q -E "High|Critical" vulnerable.txt
```

Now CI fails if a high or critical advisory appears in anything the solution uses, including packages nobody chose directly.

5.4 Semantic Versioning for a .NET solution

The version lives in `Directory.Build.props` and is stamped into every assembly. `CHANGELOG.md` defines what counts as breaking for each snack, because "public API" means different things in each:

Snack	Its contract	A breaking change, for example
1. Camera Monitor	The app itself	requiring a newer Windows or .NET
2. Access Event Log	The database schema; IAccessEventRepository (Snack 4 uses it)	a column existing databases don't have; a renamed method
3. Plugin framework	SolutionFramework.Abstractions	a new member on ISolutionModule: every module stops loading
4. Web Dashboard	The HTTP API	a renamed route, parameter or JSON field

Snack 3 is the clearest case. The contract assembly is a public API for every module ever written. Adding a method to `ISolutionModule` looks like a small, additive change, but every existing module fails to load. That's a major version. A default interface method, or a second optional interface, keeps it minor.

5.5 The workflow

YAML ../.github/workflows/genetec.yml, lines 3–15

```yaml
name: genetec

on:
  push:
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

No path filters: every push to any branch and every pull request runs the checks, so branch protection's required checks always report.

YAML ../.github/workflows/genetec.yml, lines 18–42

```yaml
  windows:
    name: build and test (Windows)
    runs-on: windows-latest
    defaults:
      run:
        working-directory: genetec
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-dotnet@v6
        with:
          global-json-file: genetec/global.json
      # --locked-mode only enforces a lock file that exists, so check every project has one.
      - name: Every project has a committed lock file
```

```
        shell: bash
        run: |
          missing=0
          for proj in $(git ls-files '*.csproj'); do
            lock="$(dirname "$proj")/packages.lock.json"
            git ls-files --error-unmatch "$lock" > /dev/null 2>&1 || { echo "::error file=$proj:
      :no committed $lock"; missing=1; }
          done
          exit $missing
      # --locked-mode: restore exactly what the packages.lock.json files record, or fail.
      - run: dotnet restore Genetec.slnx --locked-mode
      - run: dotnet build Genetec.slnx --no-restore -c Release -warnaserror
      - run: dotnet test --solution Genetec.slnx --no-build -c Release
```

Line 20 The WPF apps only build on Windows. (Snack 1's sets `EnableWindowsTargeting` and could compile on Linux; the .NET Framework one can't.)

Line 28 CI installs the SDK `global.json` asks for, so CI and your machine agree.

Line 41 Warnings fail the build. This caught xUnit's analyzer rule `xUnit1051` (pass the test's cancellation token to async calls) and the compiler's CS9124 (a constructor parameter captured twice) before they reached `main`.

YAML ../.github/workflows/genetec.yml, lines 49–75

```
  sqlserver:
    name: SQL Server tests (Linux)
    # Windows runners can't run the SQL Server container, so the database tests run here.
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: genetec
    services:
      sqlserver:
        image: mcr.microsoft.com/mssql/server:2022-latest
        env:
          ACCEPT_EULA: "Y"
          MSSQL_SA_PASSWORD: Genetec-ci-Passw0rd
        ports: ["1433:1433"]
        options: >-
          --health-cmd "/opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P Genetec-ci-Passw0rd
      -C -Q 'SELECT 1' || exit 1"
          --health-interval 10s --health-timeout 5s --health-retries 12 --health-start-period 20
      s
    env:
      TEST_SQLSERVER: Server=localhost,1433;User Id=sa;Password=Genetec-ci-Passw0rd;TrustServerC
      ertificate=True
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-dotnet@v6
        with:
          global-json-file: genetec/global.json
      - run: dotnet restore snack2-access-event-log/tests/AccessLog.Tests --locked-mode
      - run: dotnet build snack2-access-event-log/tests/AccessLog.Tests --no-restore -c Release
      -warnaserror
      - run: dotnet test --project snack2-access-event-log/tests/AccessLog.Tests --no-build -c R
      elease
```

Windows runners can't run Linux containers, so SQL Server's tests run on Linux. GitHub starts `mcr.microsoft.com/mssql/server` as a **service container** beside the job, waits for its health check, and the tests reach it on `localhost:1433`. On the Windows job the same tests are reported as skipped, which is also how they behave on a laptop without SQL Server.

What the SQL Server job caught

- **EXEC can't call a function.** `EXEC(N'CREATE DATABASE ' + QUOTENAME(@db))` is a syntax error; the statement must be built in a variable first. Every database test failed until it was fixed.
- **Error 547 means two things.** SQL Server reports foreign-key and CHECK violations with the same number. The repository turned both into "unknown device". The contract test for the CHECK rule, run on both stores, showed the in-memory store and SQL Server disagreeing.
- **A dozen rows don't need an index.** The query-plan test first ran on a nearly empty table, where reading everything is the cheapest plan, so SQL Server ignored the index. The test now inserts 20,000 rows first.

None of these would show up with the in-memory store alone. That's the argument for running the contract against the real database in CI.

5.6 Dependabot

YAML ../.github/dependabot.yml, lines 118–170

```yaml
  # ---- genetec ----
  - package-ecosystem: nuget
    directories:
      - "/genetec/snack1-camera-status-monitor/src/CameraMonitor.App"
      - "/genetec/snack1-camera-status-monitor/src/CameraMonitor.Core"
      - "/genetec/snack1-camera-status-monitor/tests/CameraMonitor.Tests"
      - "/genetec/snack2-access-event-log/src/AccessLog.App"
      - "/genetec/snack2-access-event-log/src/AccessLog.Core"
      - "/genetec/snack2-access-event-log/tests/AccessLog.Tests"
      - "/genetec/snack3-plugin-framework/modules/AfterHoursAutoLock"
      - "/genetec/snack3-plugin-framework/modules/DoorForcedEmail"
      - "/genetec/snack3-plugin-framework/src/SolutionFramework.Abstractions"
      - "/genetec/snack3-plugin-framework/src/SolutionFramework.Common"
      - "/genetec/snack3-plugin-framework/src/SolutionFramework.Host"
      - "/genetec/snack3-plugin-framework/tests/SolutionFramework.Tests"
      - "/genetec/snack4-web-dashboard/api/Dashboard.Api"
      - "/genetec/snack4-web-dashboard/tests/Dashboard.Api.Tests"
    schedule:
      interval: weekly
      day: monday
    commit-message:
      prefix: "genetec"
    groups:
      # xUnit's packages are released together.
      xunit:
        patterns: ["xunit*"]
      minor-and-patch:
        update-types: [minor, patch]
    ignore:
      # Microsoft's System.* and Microsoft.* packages follow the .NET version: 10.x goes with
      # net10.0. A major (11.x) means moving the whole solution to .NET 11, done by hand.
      - dependency-name: "System.*"
        update-types: ["version-update:semver-major"]
      - dependency-name: "Microsoft.*"
        update-types: ["version-update:semver-major"]

  - package-ecosystem: npm
    directory: "/genetec/snack4-web-dashboard/web"
    schedule:
      interval: weekly
      day: monday
    commit-message:
      prefix: "genetec"
    groups:
      react:
```

```
163         patterns: ["react", "react-dom", "@types/react", "@types/react-dom"]
164       vite:
165         patterns: ["vite", "@vitejs/*"]
166       minor-and-patch:
167         update-types: [minor, patch]
168     ignore:
169       - dependency-name: "@types/node"
170         update-types: ["version-update:semver-major"]
```

NuGet updates for every Genetec project, with xUnit's packages grouped (they release together), and npm updates for Snack 4's page, with Vite and its React plugin grouped (each new major needs the other). Each update arrives as a pull request, and the workflow above tests it before anyone merges.

The `ignore` rules skip major versions of Microsoft's `System.*` and `Microsoft.*` packages. Those follow the .NET version: 10.x goes with `net10.0`. Dependabot's first run offered `System.Security.Cryptography.Pkcs` 11.0.0 for Snack 2, the package from 5.3. It updated one project's lock file but not the ones that reference it, so `--locked-mode` failed everywhere else, which is the lock check doing its job. 10.0.x already has the fix, so the pull request was closed, and moving to .NET 11 stays a deliberate change for the whole solution.

5.7 Running it all

Snack	Run	Test
1	`dotnet run --project snack1-camera-status-monitor/src/CameraMonitor.App`	`dotnet test --project snack1-.../tests/CameraMonitor.Tests`
2	`docker compose up -d` in snack2, then `dotnet run --project src/AccessLog.App`	set TEST_SQLSERVER to include SQL Server
3	`dotnet run --project src/SolutionFramework.Host` (after a build)	`dotnet test --project tests/SolutionFramework.Tests`
4	`dotnet run --project api/Dashboard.Api`; `npm run dev` in web/	`dotnet test`; `npm test` in web/
All	`dotnet build` in genetec/	`dotnet test` in genetec/

5.8 Interview questions

What does packages.lock.json add if the .csproj already has exact versions?

The transitive graph. The project file pins only direct dependencies; the lock records every package below them and its content hash, so restore can't silently pick a different transitive version or a tampered package.

A package you never referenced has a vulnerability. How do you fix it?

Find who brings it in (`dotnet list package --include-transitive`), then either update the direct dependency to a release that depends on a fixed version, or reference the fixed version directly, which NuGet's lowest-applicable-version rule makes win.

Why does the solution build with -warnaserror in CI but not locally?

Locally, a warning is information while you work; on the way into main, it's a defect. Failing CI on warnings stops them piling up until nobody reads them.

Why test against a real SQL Server when there's an in-memory repository?

The in-memory store tests the logic above the database; only the real one tests the SQL, the constraints and the indexes. Three real bugs in this snack only showed up there.